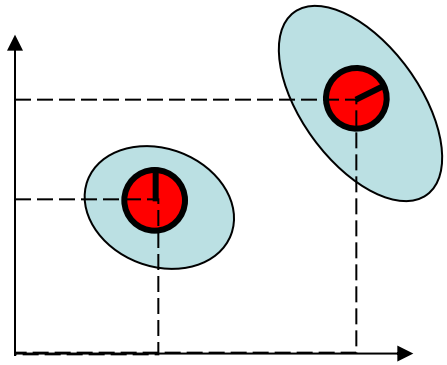
Gaussian Filters

- **The hypothesis:** Uncertainty in robot pose can be represented through a gausian random vector (grv)



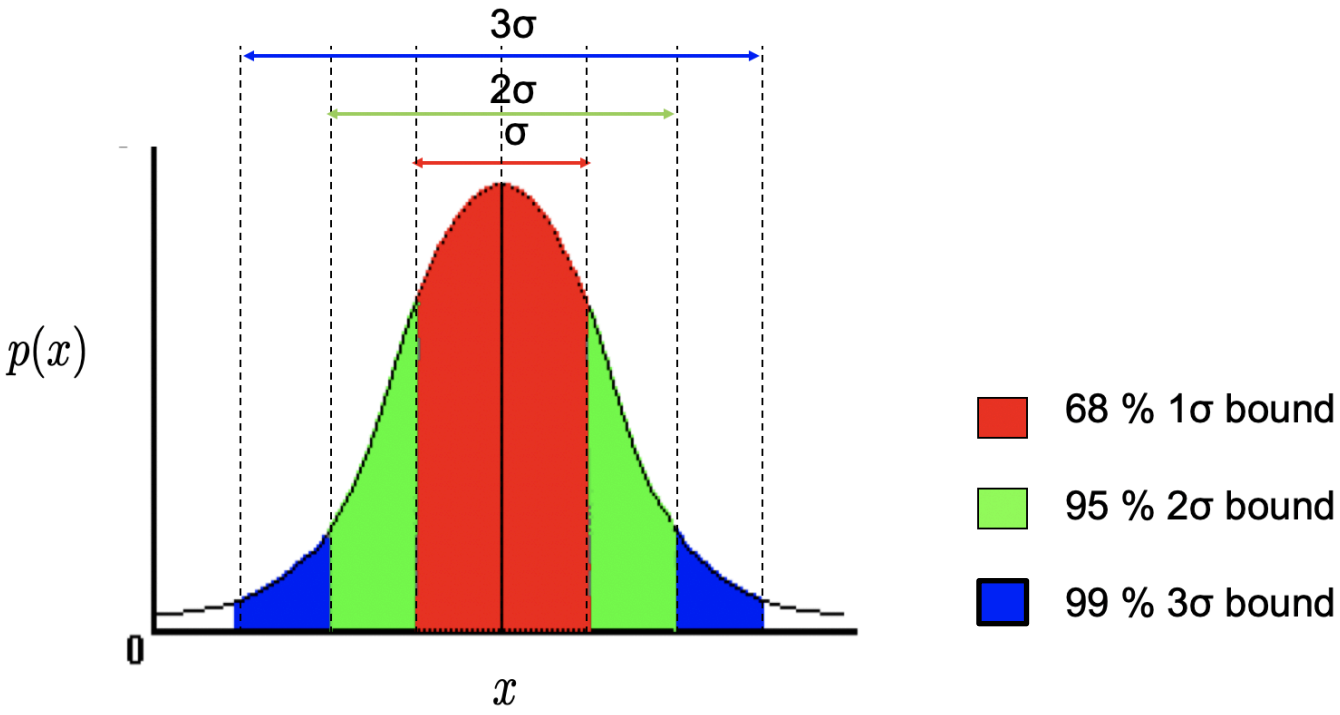
Position corrupted with Random Gaussian noise

- **Why the Gaussian hypothesis is so important?**
  - **Simplicity:** With only two parameters the pdf is completely defined.
  - **Computational efficiency:** Make some problems tractable.
  - **Central Limit Theorem (CLT):** The sum of a sequence of n independent random variables tends to a Gaussian pdf as $n \rightarrow \infty$

Gaussian Random Variables

The pdf of a (scalar) Gaussian random variable is given by:

$$f_x(x) = \mathcal{N}(\hat{x}, \sigma_x^2) = \frac{1}{\sigma_x \sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\hat{x}}{\sigma_x}\right)^2}$$



Gaussian Random Vectors (GRV)

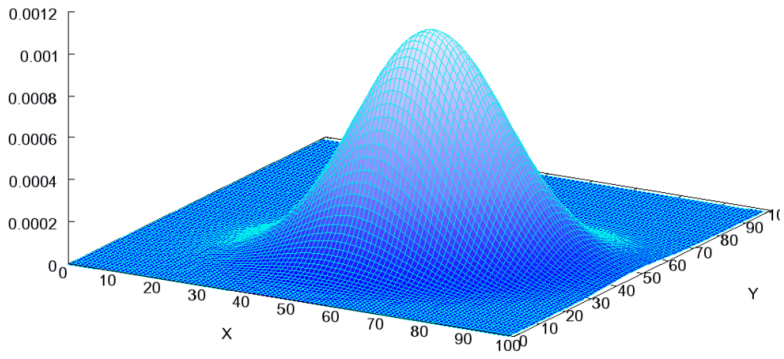
A Gaussian Random Vector has the following pdf:

$$\mathcal{N}(\hat{\boldsymbol{x}}, \boldsymbol{P}) = (2\pi)^{-\frac{n}{2}} \cdot |\boldsymbol{P}|^{-\frac{1}{2}} \cdot e^{-\frac{1}{2}(\boldsymbol{x}-\hat{\boldsymbol{x}})^T \boldsymbol{P}^{-1}(\boldsymbol{x}-\hat{\boldsymbol{x}})} \tag{1}$$

where:

$$\hat{\boldsymbol{x}} = \mathbb{E}[\boldsymbol{x}] = [\hat{x}_1 \ \hat{x}_2 \ \dots \ \hat{x}_n]^T$$

$$\boldsymbol{P} = \mathbb{E}[(\boldsymbol{x} - \hat{\boldsymbol{x}}) \cdot (\boldsymbol{x} - \hat{\boldsymbol{x}})^T] = \begin{bmatrix} \sigma_{11}^2 & \sigma_{12} & \dots & \sigma_{1n} \\ \sigma_{21}^2 & \sigma_{22}^2 & \dots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1}^2 & \sigma_{n2} & \dots & \sigma_{nn}^2 \end{bmatrix} \tag{2}$$



σ_i^2: *Variance of component i*

σ_{ij}: *Covariance between component i & j*

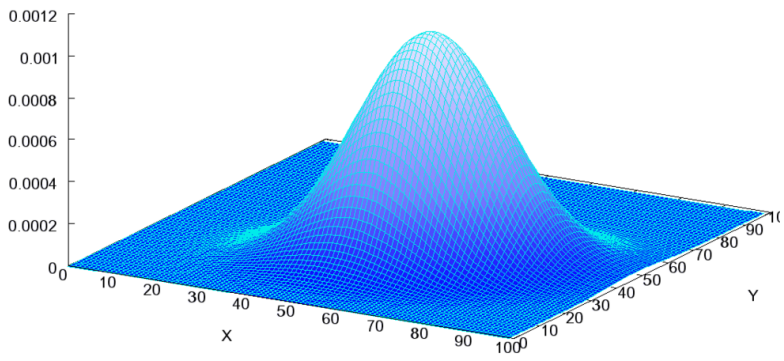
Gaussian Random Vectors (GRV)

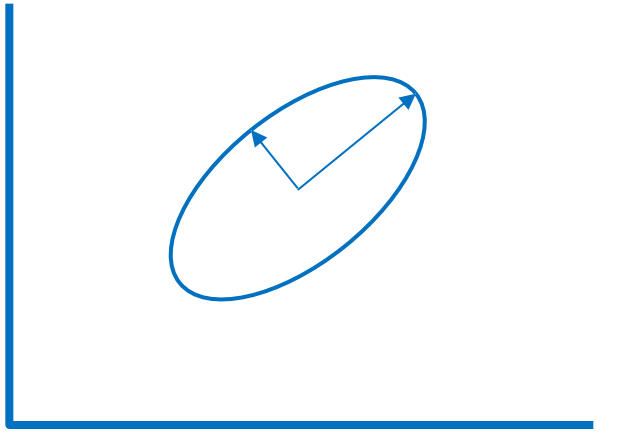
Properties:

1. $\boldsymbol{P} = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^T] - \hat{\boldsymbol{x}}\hat{\boldsymbol{x}}^T$
2. $\boldsymbol{P}$ **is symmetric**: $\boldsymbol{P} = \boldsymbol{P}^T \Rightarrow \sigma_{ij} = \sigma_{ji}, \forall i, j \in \mathbb{N} / 1 \leq i, j \leq n$

   $\Rightarrow$ Eigenvectors are orthogonal (ellipse semiaxes)

   $\Rightarrow$ Eigenvalues are positive (length of the semiaxes)
3. $\boldsymbol{P}$ **is positive-semidefinite**: $\boldsymbol{P} \geq \boldsymbol{0} \Rightarrow \boldsymbol{a}^{\boldsymbol{T}} \cdot \boldsymbol{P} \cdot \boldsymbol{a} \geq \boldsymbol{0},\ \forall \boldsymbol{a} \in \mathbb{R}^n$





Semiaxes length = Eigenvalue $^{1/2}$
Semiaxes = Eigenvectors

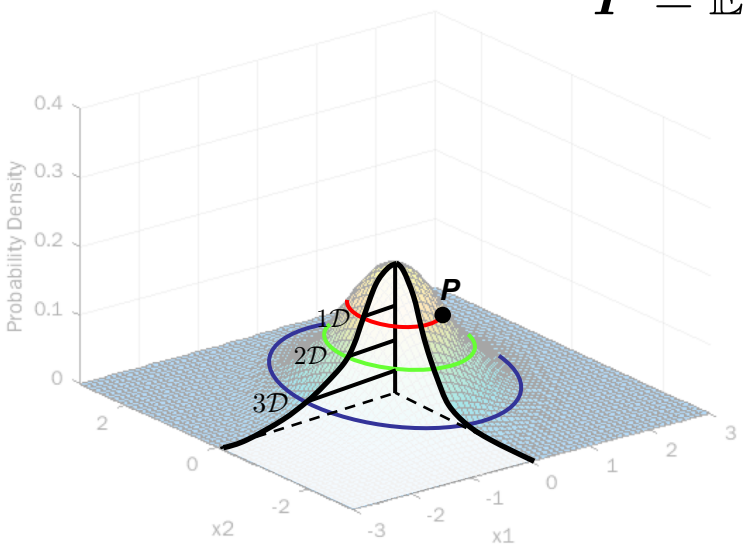
Gaussian Random Vectors (GRV)

A Gaussian Random Vector has the following pdf:

$$\mathcal{N}(\hat{\boldsymbol{x}}, \boldsymbol{P}) = (2\pi)^{-\frac{n}{2}} \cdot |\boldsymbol{P}|^{-\frac{1}{2}} \cdot e^{-\frac{1}{2}(\boldsymbol{x}-\hat{\boldsymbol{x}})^T \boldsymbol{P}^{-1}(\boldsymbol{x}-\hat{\boldsymbol{x}})} \tag{1}$$

where:

$$\begin{aligned} \hat{\boldsymbol{x}} &= \mathbb{E}[\boldsymbol{x}] = [\hat{x}_1 \ \hat{x}_2 \ \ldots \ \hat{x}_n]^T \\ \boldsymbol{P} &= \mathbb{E}[(\boldsymbol{x} - \hat{\boldsymbol{x}}) \cdot (\boldsymbol{x} - \hat{\boldsymbol{x}})^T] \end{aligned} \tag{2}$$



Confidence Intervals

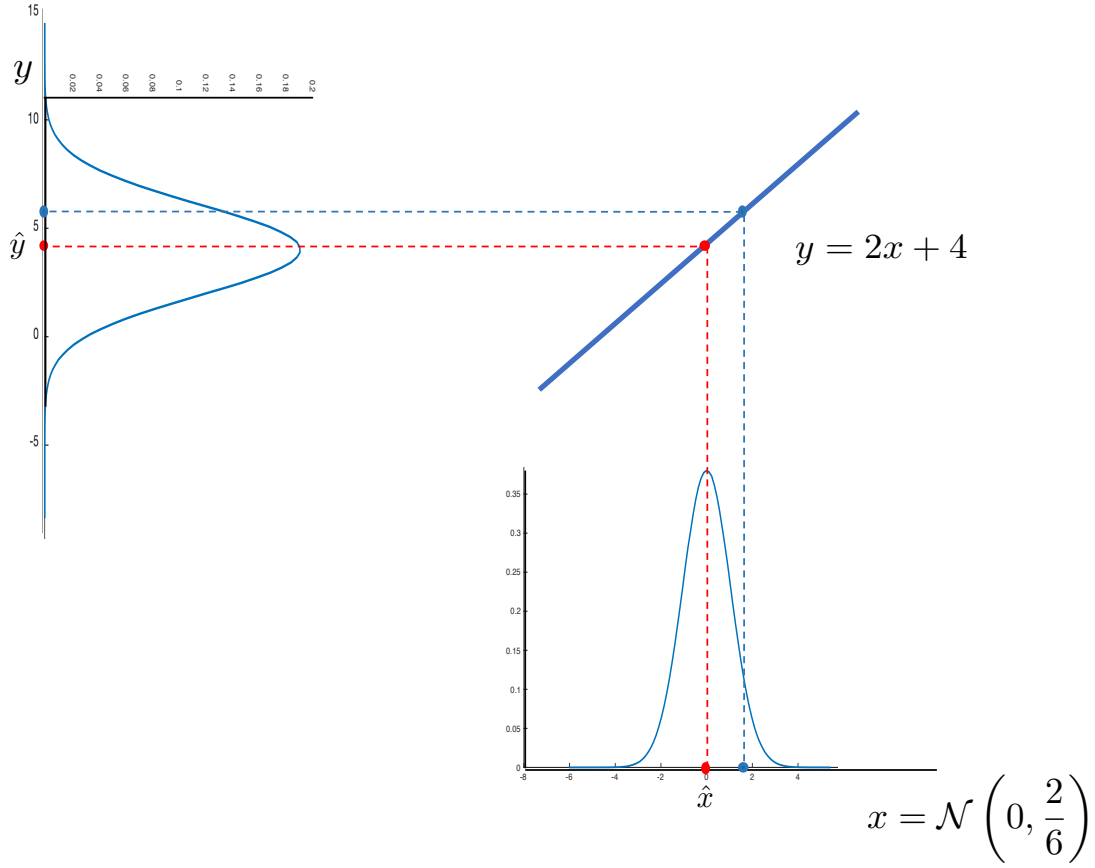
$$P(\boldsymbol{x} \in \mathcal{E}_{\mathcal{D}_1}) = \int_{\mathcal{E}_{\mathcal{D}_1}} f_x(x)dx = 0.4$$

$$P(\boldsymbol{x} \in \mathcal{E}_{\mathcal{D}_{2.45}}) = \int_{\mathcal{E}_{\mathcal{D}_{2.45}}} f_x(x)dx = 0.95$$

$$P(\boldsymbol{x} \in \mathcal{E}_{\mathcal{D}_{3.03}}) = \int_{\mathcal{E}_{\mathcal{D}_{3.03}}} f_x(x)dx = 0.99$$

- ***Mahalanobis distance:*** *multi-dimensional generalization of the idea of measuring how many standard deviations away P is from the mean*

Properties: Linear transformations of GRV



- We stay in the “Gaussian world” as long as we start with Gaussians and perform only linear transformations.

Table 1.1: Linear transformation of a Gaussian variable

Scalar	$x \sim \mathcal{N}(\hat{x}, \sigma_x^2)$ $y = ax + b$	$\Rightarrow y \sim \mathcal{N}(a\hat{x} + b, a^2\sigma_x^2)$
	$x \sim \mathcal{N}(\hat{x}, \sigma_x^2)$ $y \sim \mathcal{N}(\hat{y}, \sigma_y^2)$ $z = ax + by$	$\Rightarrow z \sim \mathcal{N}(a\hat{x} + b\hat{y}, a^2\sigma_x^2 + 2ab\sigma_{xy^2} + b^2\sigma_y^2)$
Vectorial	$\boldsymbol{x} \sim \mathcal{N}(\hat{\boldsymbol{x}}, \boldsymbol{P_x})$ $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$	$\Rightarrow \boldsymbol{y} \sim \mathcal{N}(\boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{b}, \boldsymbol{A}\boldsymbol{P_x}\boldsymbol{A}^{\boldsymbol{T}})$
	$\boldsymbol{x} \sim \mathcal{N}(\hat{\boldsymbol{x}}, \boldsymbol{P_x})$ $\boldsymbol{y} \sim \mathcal{N}(\hat{\boldsymbol{y}}, \boldsymbol{P_y})$ $\boldsymbol{z} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{y}$	$\Rightarrow \boldsymbol{z} \sim \mathcal{N}(\boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}\hat{\boldsymbol{y}}, \boldsymbol{A}\boldsymbol{P_x}\boldsymbol{A}^{\boldsymbol{T}} + \boldsymbol{A}\boldsymbol{P_{xy}}\boldsymbol{B}^{\boldsymbol{T}} + \boldsymbol{B}\boldsymbol{P_{yx}}\boldsymbol{A}^{\boldsymbol{T}} + \boldsymbol{B}\boldsymbol{P_y}\boldsymbol{B}^{\boldsymbol{T}})$

Table 1.2: Operations with Gaussian PDFs.

Scalar	$x \sim \mathcal{N}(\hat{x}, \sigma_x^2)$ $y \sim \mathcal{N}(\hat{y}, \sigma_y^2)$ $z = xy$	$\Rightarrow z \sim \mathcal{N}\left(\frac{\sigma_y^2}{\sigma_x^2+\sigma_y^2}\hat{x} + \frac{\sigma_x^2}{\sigma_x^2+\sigma_y^2}\hat{y}, \frac{1}{\sigma_x^{-2}+\sigma_y^{-2}}\hat{x}\right)$
Vectorial	$\boldsymbol{x} \sim \mathcal{N}(\hat{\boldsymbol{x}}, \boldsymbol{P_x})$ $\boldsymbol{y} \sim \mathcal{N}(\hat{\boldsymbol{y}}, \boldsymbol{P_y})$ $\boldsymbol{z} = \boldsymbol{x}\boldsymbol{y}$	$\Rightarrow \boldsymbol{z} \sim \mathcal{N}\left(\frac{\boldsymbol{P_y}}{\boldsymbol{Px}+\boldsymbol{Py}}\hat{\boldsymbol{x}} + \frac{\boldsymbol{P_x}}{\boldsymbol{Px}+\boldsymbol{Py}}\hat{\boldsymbol{y}}, \frac{\boldsymbol{I}}{\boldsymbol{Px}^{-1}+\boldsymbol{Py}^{-1}}\right)$

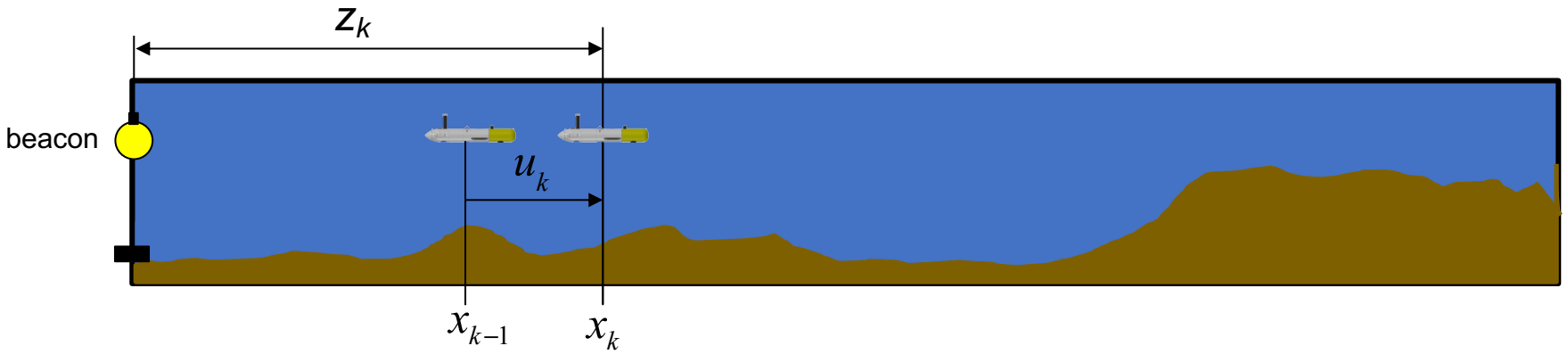
Linear Stochastic Systems

$$\begin{aligned} \bar{\boldsymbol{x}}_{\boldsymbol{k}} &= \boldsymbol{A_k x_{k-1}} + \boldsymbol{B_k u_k} + \boldsymbol{w_k} \\ \boldsymbol{z_k} &= \boldsymbol{H_k} \bar{\boldsymbol{x}}_{\boldsymbol{k}} + \boldsymbol{v_k} \end{aligned} \tag{1}$$

where:

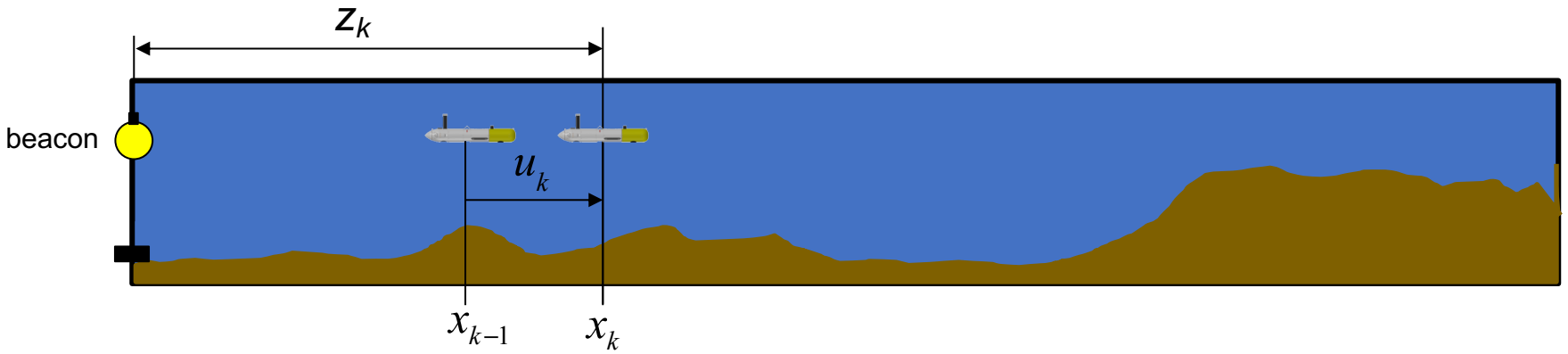
- $\boldsymbol{A_k}$: Matrix $(n \times n)$ that describes how the state evolves from time k to $k-1$ without controls or noise.
- $\boldsymbol{B_k}$: Matrix $(n \times l)$ that describes how the control $\boldsymbol{u_k}$ changes the state from k to $k-1$
- $\boldsymbol{H_k}$: Matrix $(k \times n)$ Describes the output $\boldsymbol{z_k}$ that would be observed from the current state $\boldsymbol{x_k}$ in absence of noise.
- $\boldsymbol{x_{k-1}}$: State Vector $(n \times 1)$. State of the system at time step $k-1$.
- $\bar{\boldsymbol{x}}_{\boldsymbol{k}}$: state Vector $(n \times 1)$ predicted by the LGS at time step k.
- $\boldsymbol{u_k}$: Input Vector $(n \times 1)$.
- $\boldsymbol{z_k}$: Output vector of the system.
- $\boldsymbol{w_k}$: Uncertainty of the dynamical model.
- $\boldsymbol{v_k}$: Uncertainty of the observation (output) model.

1 DOF AUV Localization with displacement and position fix



1. t_{k-1} : The AUV is at position x_{k-1}
2. t_k: The AUV Makes a displacement u_k
3. t_k: The AUV Senses the distance wrt to the beacon

1 DOF AUV Localization with displacement and position fix



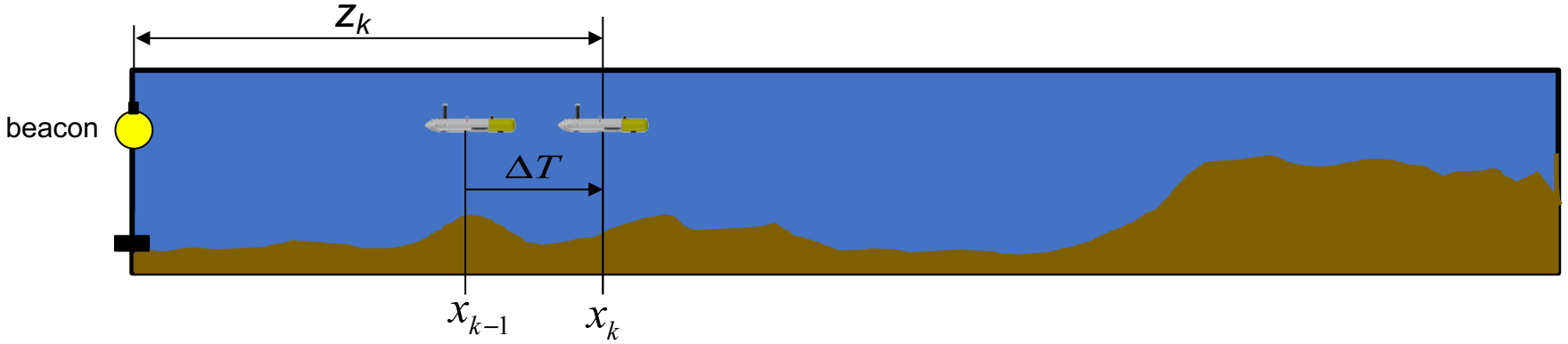
1. t_{k-1} : The AUV is at position x_{k-1}
2. t_k: The AUV Makes a displacement u_k
3. t_k: The AUV Senses the distance wrt to the beacon

$$x_k = x_{k-1} + u_k + w_k$$
$$z_k = x_k + v_k$$

$$A_k = 1$$
$$B_k = 1$$
$$H_k = 1$$
$$w_k = N(0,\sigma_w)$$
$$v_k = N(0,\sigma_v)$$

$$x_k = A_k x_{k-1} + B_k u_k + w_k$$
$$z_k = H_k x_k + v_k$$

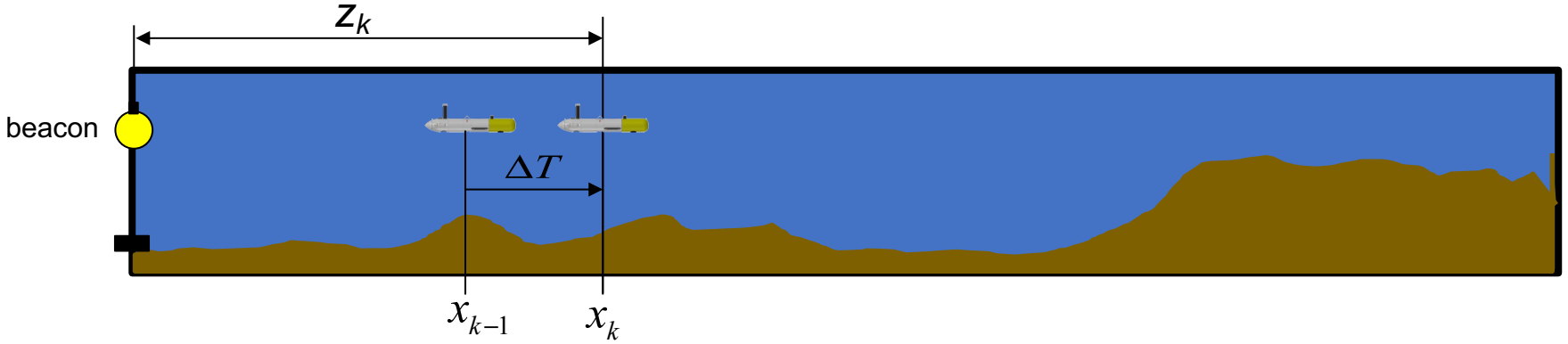
1DOF AUV Navigation with Ct Velocity Model & Velocity updates



1. t_{k-1} : The AUV is at position x_{k-1}
2. t_{k-1}: The AUV Moves at ct. velocity $\dot{x}_{k-1}$
3. t_k: The AUV reaches position x_k
4. t_k: The AUV measures its velocity z_k

Gaussian Filters

1DOF AUV Navigation with Ct Velocity Model & Velocity updates



1. t_{k-1} : The AUV is at position x_{k-1}
2. t_{k-1}: The AUV Moves at ct. velocity $\dot{x}_{k-1}$
3. t_k: The AUV reaches position x_k
4. t_k: The AUV measures its velocity z_k

$$a_k = w_k \sim \mathcal{N}(0, \sigma_w^2)$$
$$\dot{\bar{x}}_k = \dot{x}_{k-1} + w_k \Delta t$$
$$\bar{x}_k = x_{k-1} + \dot{x}_{k-1}\Delta t + \frac{\Delta t^2}{2} w_k$$
$$z_k = \dot{\bar{x}}_k + m \cdot v_k$$

$$\underbrace{\begin{bmatrix} x_k \\ \dot{x}_k \end{bmatrix}}_{\bar{\boldsymbol{x}}_k} = \underbrace{\begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}}_{\boldsymbol{A_k}} \underbrace{\begin{bmatrix} x_{k-1} \\ \dot{x}_{k-1} \end{bmatrix}}_{\boldsymbol{x_{k-1}}} + \underbrace{\begin{bmatrix} \frac{\Delta t^2}{2} \\ \Delta t \end{bmatrix}}_{\boldsymbol{w_k}} w_k$$

$$z_k = \underbrace{\begin{bmatrix} 0 & 1 \end{bmatrix}}_{\boldsymbol{H_k}} \underbrace{\begin{bmatrix} x_k \\ \dot{x}_k \end{bmatrix}}_{\bar{\boldsymbol{x}}_k} + v_k$$

Gaussian Filters

The Kalman Filter

Algorithm Bayes_filter$(bel(x_{t-1}), u_t, z_t)$**:**

for all x_t do

$$\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1})\, bel(x_{t-1})\, dx$$

$$bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)$$

endfor

return $bel(x_t)$

Algorithm Kalman Filter $(\hat{x}_{k-1}, P_{k-1}, u_k, z_k)$

$$\hat{x}_k^- = A_k \hat{x}_{k-1} + B_k u_k$$

$$P_k^- = A_k P_{k-1} A_k^T + Q_k$$

$$K_k = P_k^- H_k^T \left(H_k P_k^- H_k^T + R_k\right)^{-1}$$

$$\hat{x}_k = \hat{x}_k^- + K_k \left(z_k - H_k \hat{x}_k^-\right)$$

$$P_k = \left(I - K_k H_k\right) P_k^-$$

$$return\ \left(\hat{x}_k, P_k\right)$$

- **Optimal implementation of the Bayes filter for:**
  - **Linear Stochastic Systems:**
    - Possibly time varying
  - **Noise (process & measurement) should be:**
    - Gaussian zero mean
    - White
    - Possibly non stationary
    - Mutually independent

    } **Mathematically Tractable**
  - **Computes de MMSE estimator for LG systems**

Gaussian Filters

1DOF AUV Navigation with Displacement input & Position update

Method `LocalizationLoop`$(\hat{x}_0, P_0)$

$[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;

for $k = 1$ *to* ∞ **do**

$[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$

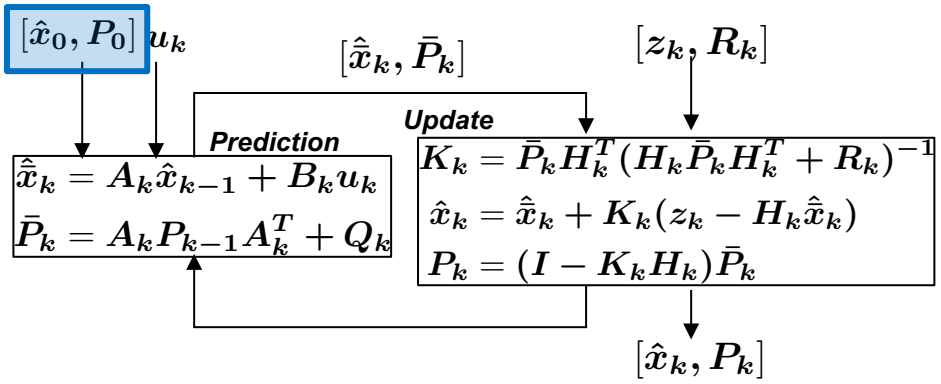
$[u_k, Q_k] = GetInput()$;

$[\bar{\hat{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;

$[z_k, R_k, H_k, V_k] = GetMeasurements()$;

$[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



1. Initial position is known with some uncertainty

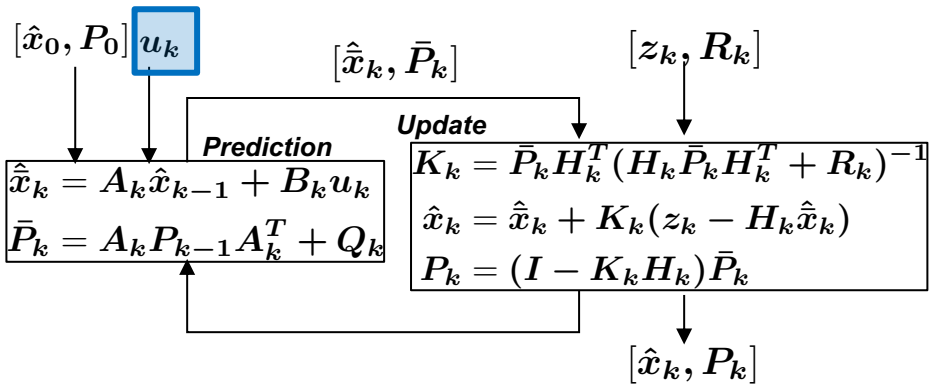
Gaussian Filters

1DOF AUV Navigation with Displacement input & Position update

Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\bar{\hat{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



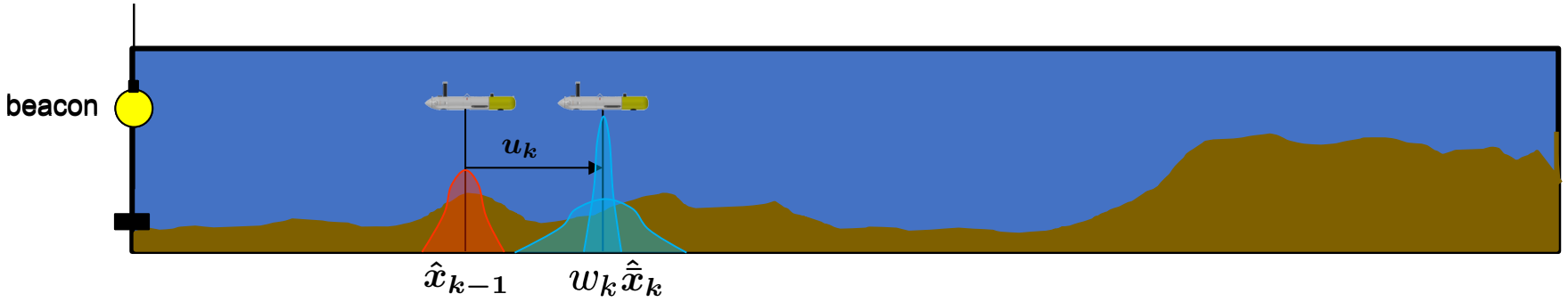
2. The robot moves and the model is used to predict the new position

Method GetInputs()
- $\dot{x}_k = GetDVL()$;
- $u_k = \dot{x}_k \Delta t$; $Q_k = \sigma^2_{DVL} \Delta t^2$

Return$([u_k, Q_k])$;

Gaussian Filters

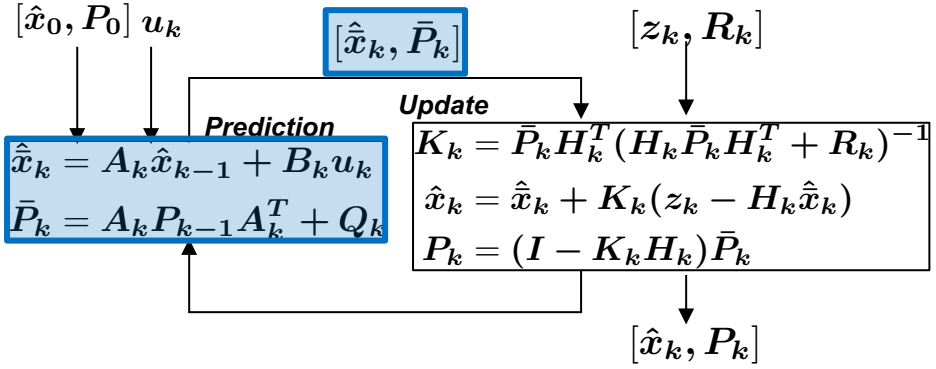
1DOF AUV Navigation with Displacement input & Position update



Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\hat{\bar{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



Motion Model

2. The robot moves and the model is used to predict the new position

Method GetInputs()
- $\dot{x}_k = GetDVL()$;
- $u_k = \dot{x}_k \Delta t$; $Q_k = \sigma^2_{DVL} \Delta t^2$

Return$([u_k, Q_k])$;

$$\bar{x}_k = x_{k-1} + u_k + w_k \;;\; w_k \sim \mathcal{N}(0, \sigma^2_{w_k})$$

$$\sigma^2_{\bar{x}_k} = \sigma^2_{x_{k-1}} + \sigma^2_{w_k}$$

$$\bar{P}_k = \sigma^2_{\bar{x}_k};\; P_k = \sigma^2_{x_k};\; P_{k-1} = \sigma^2_{x_{k-1}};\; A_k = 1;\; B_k = 1;$$

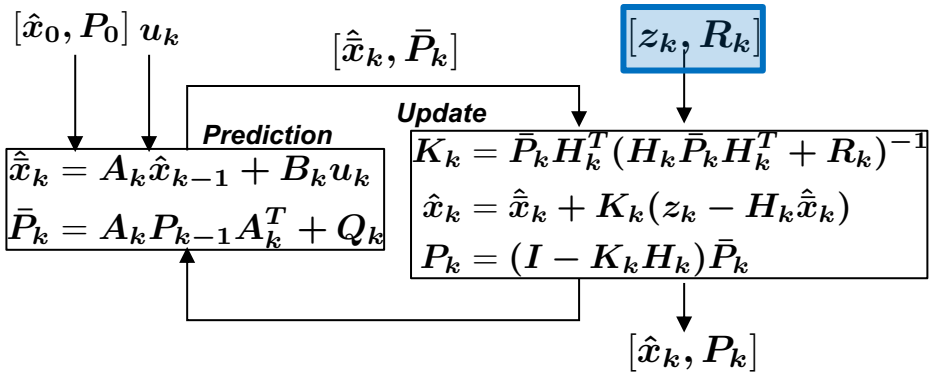
Gaussian Filters

1DOF AUV Navigation with Displacement input & Position update

Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\hat{\bar{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



3. The robot position is sensed

Method GetMeasurements()
- $z_k = GetRange()$;
- $R_k = \sigma^2_{Range}$
- $H_k = 1$

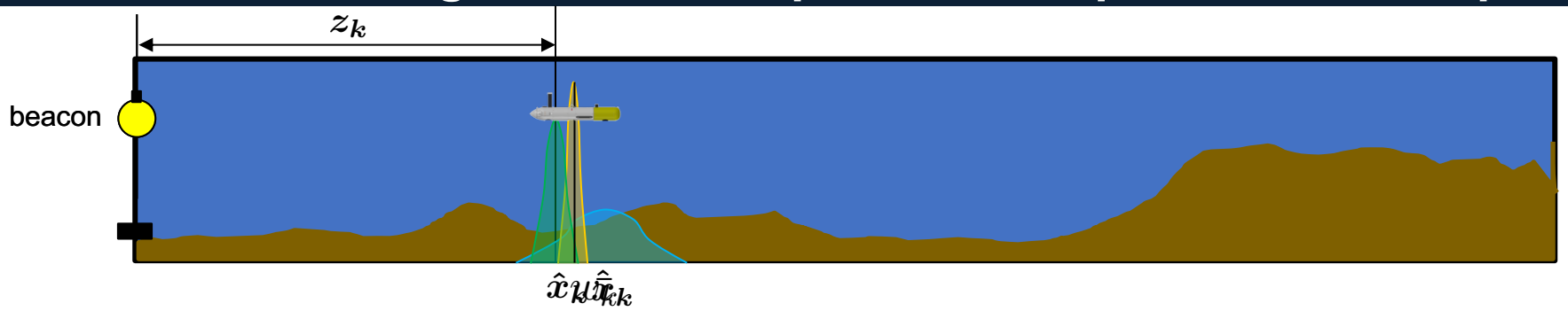
Return$([z_k, R_k, Hk])$;

$$z_k = x_k + v_k \;;\; v_k \sim \mathcal{N}(0, \sigma^2_{v_k})$$

$$H_k = 1 \;;\; \boldsymbol{R_k} = \sigma^2_{v_k}$$

Gaussian Filters

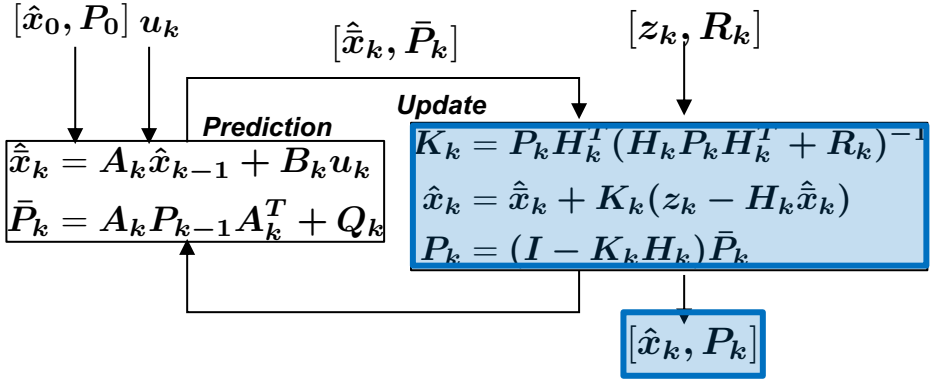
1DOF AUV Navigation with Displacement input & Position update



Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\bar{\hat{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, P_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$

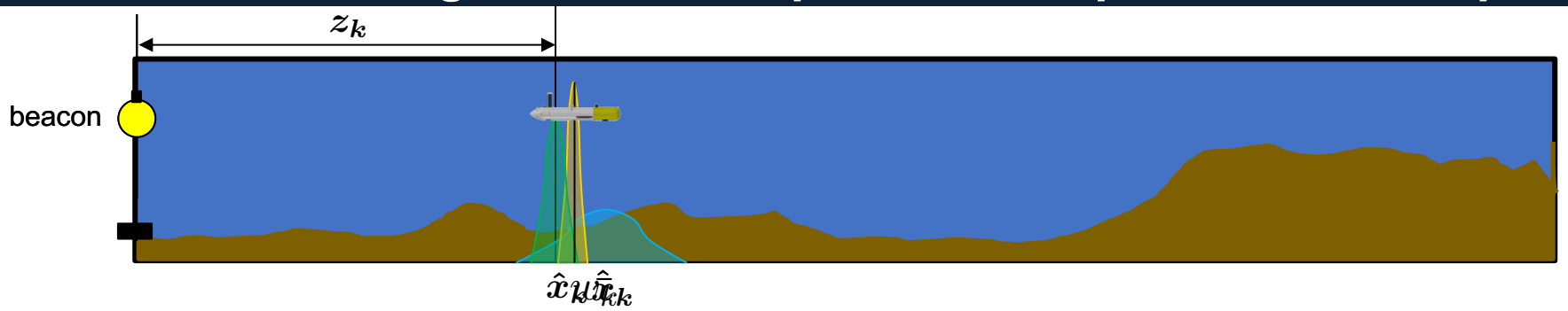


4. The measured position is used to update the robot position

$$\left.\begin{aligned} \sigma^2_{\bar{x}_k} &= 0 \Rightarrow K_k = 0 \\ K_k &= \frac{\sigma^2_{\bar{x}_k}}{\sigma^2_{\bar{x}_k} + \sigma^2_{v_k}} \\ \hat{x}_k &= \bar{\hat{x}}_k + K_k(z_k - \bar{\hat{x}}_k) \\ \sigma^2_{x_k} &= \frac{\sigma^2_{\bar{x}_k}\sigma^2_{v_k}}{\sigma^2_{\bar{x}_k} + \sigma^2_{v_k}} = 0 \end{aligned}\right\} \Rightarrow \hat{x}_k = \bar{\hat{x}}_k$$

Gaussian Filters

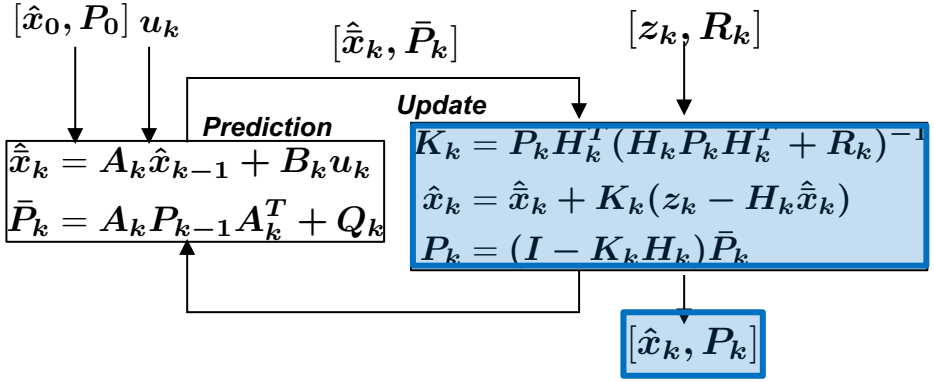
1DOF AUV Navigation with Displacement input & Position update



Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\hat{\bar{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$

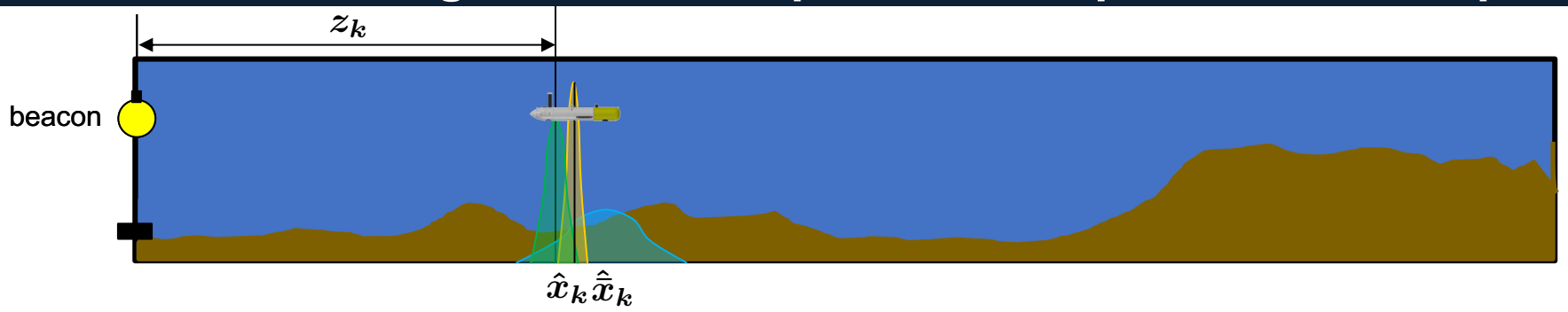


4. The measured position is used to update the robot position

$$\left.\begin{aligned} \sigma_{v_k}^2 &= 0 \Rightarrow K_k = 1 \\ K_k &= \frac{\sigma_{\bar{x}_k}^2}{\sigma_{\bar{x}_k}^2 + \sigma_{v_k}^2} \\ \hat{x}_k &= \hat{\bar{x}}_k + K_k(z_k - \hat{\bar{x}}_k) \\ \sigma_{x_k}^2 &= \frac{\sigma_{\bar{x}_k}^2 \sigma_{v_k}^2}{\sigma_{\bar{x}_k}^2 + \sigma_{v_k}^2} = 0 \end{aligned}\right\} \Rightarrow \hat{x}_k = z_k$$

Gaussian Filters

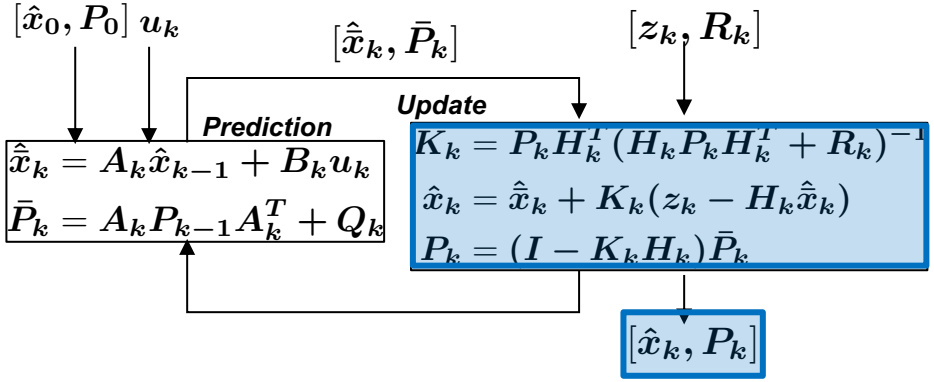
1DOF AUV Navigation with Displacement input & Position update



Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\bar{\hat{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, P_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



How Update Works

4. The measured position is used to update the robot position

$$\left.\begin{aligned} K_k &= \frac{\sigma^2_{\bar{x}_k}}{\sigma^2_{\bar{x}_k} + \sigma^2_{v_k}} \\ \hat{x}_k &= \bar{\hat{x}}_k + K_k(z_k - \bar{\hat{x}}_k) \\ \sigma^2_{x_k} &= \frac{\sigma^2_{\bar{x}_k}\sigma^2_{v_k}}{\sigma^2_{\bar{x}_k} + \sigma^2_{v_k}};\ \sigma^2_{x_k} < \sigma^2_{\bar{x}_k}, \sigma^2_{x_k} < \sigma^2_{v_k} \end{aligned}\right\} \Rightarrow \hat{x}_k \approx \frac{\sigma^2_{v_k}}{\sigma^2_{\bar{x}_k} + \sigma^2_{v_k}} \bar{\hat{x}}_k + \frac{\sigma^2_{\bar{x}_k}}{\sigma^2_{\bar{x}_k} + \sigma^2_{v_k}} z_k$$

Gaussian Filters

Implementation

Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop(`$\hat{\boldsymbol{x}}_{\boldsymbol{0}}, \boldsymbol{P_0}$`)`
  - $[\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}] = [\hat{\boldsymbol{x}}_{\boldsymbol{0}}, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k}] = Localize(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}})$
- **Method** `Localize(`$\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}$`)`
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}}, \bar{\boldsymbol{P}}_{\boldsymbol{k}}] = Prediction(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k}]$

Class *KF : GaussianFilter*

- **Constructor** `KF(`$\boldsymbol{A_k}, \boldsymbol{B_k}$`)`
  - $self.\boldsymbol{A_k} = \boldsymbol{A_k}$;
  - $self.\boldsymbol{B_k} = \boldsymbol{B_k}$;
- **Method** `Prediction(`$\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P}_{k-1}, \boldsymbol{u}_k, \boldsymbol{Q}_k$`)`
  - $\hat{\bar{\boldsymbol{x}}}_k = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{B}_k \boldsymbol{u}_k$;
  - $\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k \boldsymbol{P}_{k-1} \boldsymbol{A}_k^T + \boldsymbol{Q}_k$;
  - **return** $\hat{\bar{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k$;
- **Method** `Update(`$\hat{\bar{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k, \boldsymbol{z}_k, \boldsymbol{R}_k, \boldsymbol{H}_k$`)`
  - $\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T (\boldsymbol{H}_k \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T + \boldsymbol{R}_k)^{-1}$;
  - $\hat{\boldsymbol{x}}_k = \hat{\bar{\boldsymbol{x}}}_k + \boldsymbol{K}_k (\boldsymbol{z}_k - \boldsymbol{H}_k \hat{\bar{\boldsymbol{x}}}_k)$;
  - $\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k) \bar{\boldsymbol{P}}_k (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k)^T + \boldsymbol{K}_k \boldsymbol{R}_k \boldsymbol{K}_k^T$;
  - **return** $\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k$;

Class *KF_1DOFAUV_InputDisplacement_PositionOM : GFLocalization, KF*

- **Constructor** `KF_1DOFAUV_InputDisplacement_PositionOM()`
  - $KF(1,1)$; `// KF constructor with` $A_k = 1$ & $B_k = 1$
- **Method** `GetInput()`
  - $\dot{x}_k = GetDVL()$; `// Velocity`
  - $u_k = \dot{x}_k \Delta t$; `// Displacement`
  - $Q_k = \sigma^2_{DVL} \Delta t^2$; `// Displacement uncertainty`
  - **return** $[\boldsymbol{u_k}\ \boldsymbol{Q_k}]$
- **Method** `GetMeasurements()`
  - $[z_k, \sigma^2_{range}] = GetRange()$;
  - $R_k = \sigma^2_{range}$;
  - **return** $[z_k, R_k]$

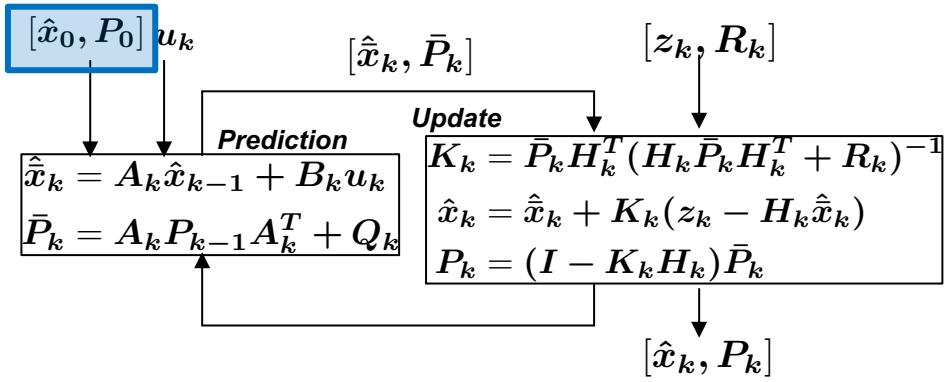
Gaussian Filters

1DOF AUV Navigation with Ct Velocity Model, & Velocity Update

Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\bar{\hat{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

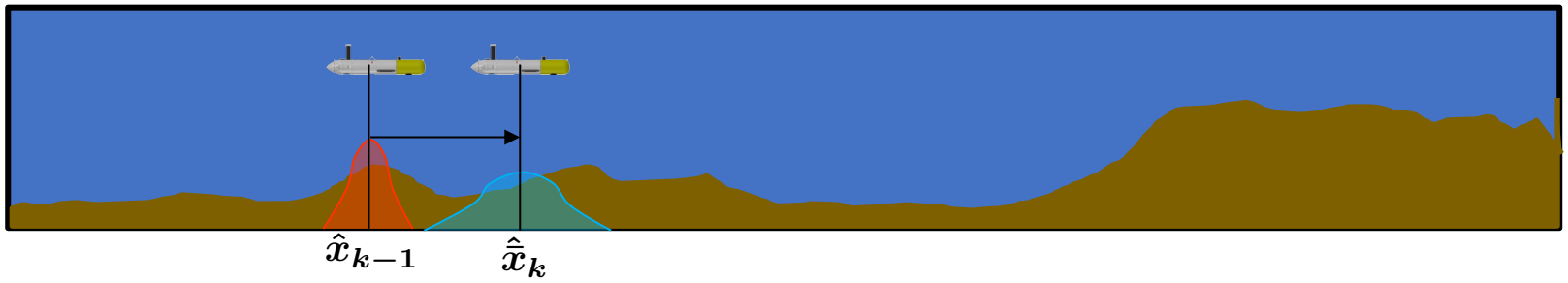
return $[\hat{x}_k, P_k]$



1. Initial position is known with some uncertainty

Gaussian Filters

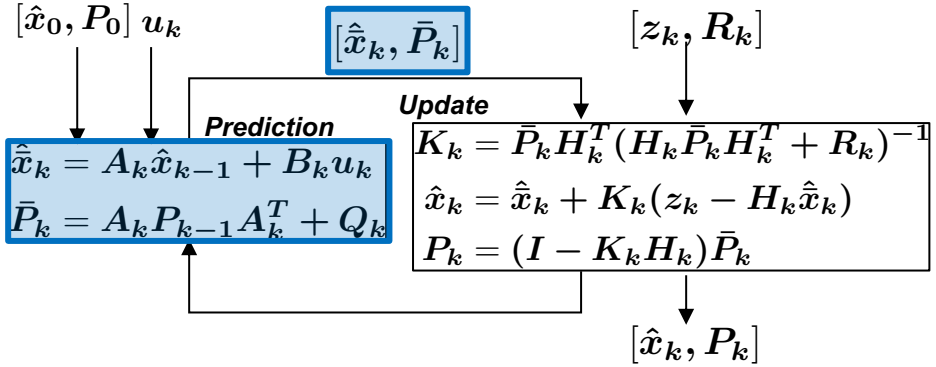
1DOF AUV Navigation with Ct Velocity Model, & Velocity Update



Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\hat{\bar{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



Motion Model

2. The robot moves and the model is used to predict the new position

$$\underbrace{\begin{bmatrix} x_k \\ \dot{x}_k \end{bmatrix}}_{\bar{x}_k} = \underbrace{\begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}}_{A_k} \underbrace{\begin{bmatrix} x_{k-1} \\ \dot{x}_{k-1} \end{bmatrix}}_{x_{k-1}} + \underbrace{\begin{bmatrix} \frac{\Delta t^2}{2} \\ \Delta t \end{bmatrix}}_{w_k} w_k \; ; \; B_k = 0; \; u_k = 0$$

$$w_k = \mathcal{N}(0, Q_k) \; ; \; Q_k = \begin{bmatrix} \frac{\Delta t^2}{2} \\ \Delta t \end{bmatrix} \sigma_w^2 \begin{bmatrix} \frac{\Delta t^2}{2} & \Delta t \end{bmatrix} = \begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 \end{bmatrix}$$

Gaussian Filters

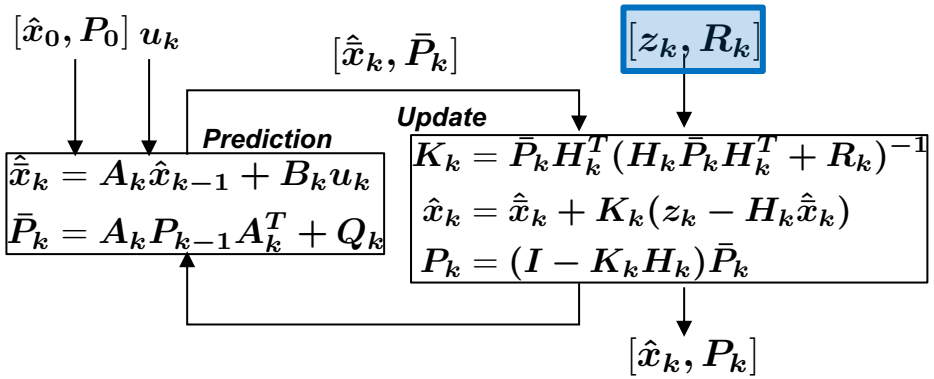
1DOF AUV Navigation with Ct Velocity Model, & Velocity Update

$\hat{\bar{x}}_k$

Method LocalizationLoop($\hat{x}_0, P_0$)
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method Localize($\hat{x}_{k-1}, P_{k-1}$)
- $[u_k, Q_k] = GetInput()$;
- $[\hat{\bar{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



Observation Model

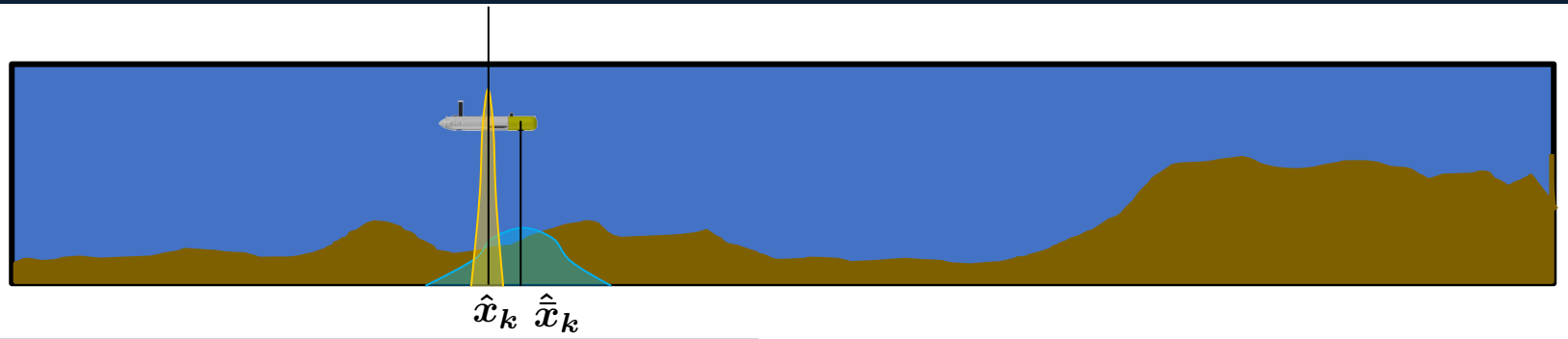
3. The robot position is sensed

Method GetMeasurements()
- $z_k = GetDVL()$;
- $R_k = \sigma^2_{DVL}$

Return($[z_k, R_k]$);

Gaussian Filters

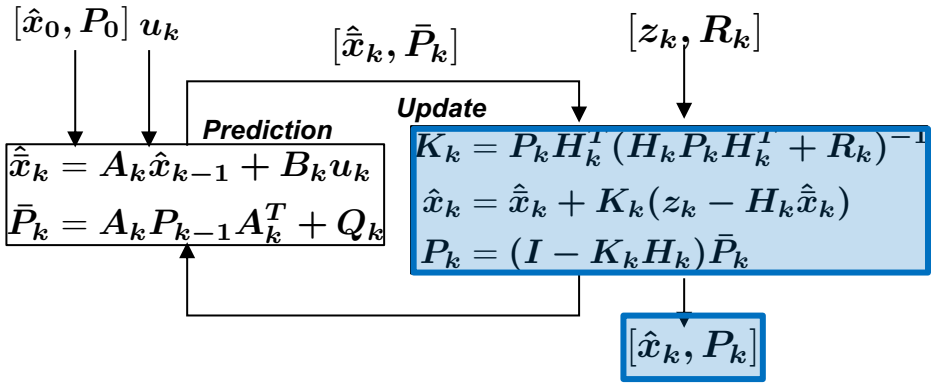
1DOF AUV Navigation with Ct Velocity Model, & Velocity Update



Method `LocalizationLoop`$(\hat{x}_0, P_0)$
- $[\hat{x}_{k-1}, P_{k-1}] = [\hat{x}_0, P_0]$;
- **for** $k = 1$ *to* ∞ **do**
  - $[\hat{x}_k, P_k] = Localize(\hat{x}_{k-1}, P_{k-1})$

Method `Localize`$(\hat{x}_{k-1}, P_{k-1})$
- $[u_k, Q_k] = GetInput()$;
- $[\hat{\bar{x}}_k, \bar{P}_k] = Prediction(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$;
- $[z_k, R_k, H_k, V_k] = GetMeasurements()$;
- $[\hat{x}_k, \bar{P}_k] = Update(\hat{x}_{k-1}, P_{k-1}, z_k, R_k, H_k, V_k)$

return $[\hat{x}_k, P_k]$



Observation Model

4. The measured position is used to update the robot position

Method GetMeasurements()
- $z_k = GetDVL()$;
- $R_k = \sigma^2_{DVL}$

Return$([z_k, R_k])$;

$$z_k = \underbrace{\begin{bmatrix} 0 & 1 \end{bmatrix}}_{H_k} \underbrace{\begin{bmatrix} x_k \\ \dot{x}_k \end{bmatrix}}_{\bar{x}_k} + v_k \;;\; v_k = \mathcal{N}(0, R_k)$$

Gaussian Filters

Implementation

Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop(`$\hat{\boldsymbol{x}}_0, \boldsymbol{P_0}$`)`
  - $[\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}] = [\hat{\boldsymbol{x}}_0, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_k, \boldsymbol{P_k}] = Localize(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
- **Method** `Localize(`$\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}$`)`
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\bar{\hat{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k] = Prediction(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_k, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_k, \boldsymbol{P_k}]$

Class *KF : GaussianFilter*

- **Constructor** `KF(`$\boldsymbol{A}_k, \boldsymbol{B}_k$`)`
  - $self.\boldsymbol{A}_k = \boldsymbol{A}_k$;
  - $self.\boldsymbol{B}_k = \boldsymbol{B}_k$;
- **Method** `Prediction(`$\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{u}_k, \boldsymbol{Q}_k$`)`
  - $\bar{\hat{\boldsymbol{x}}}_k = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{B}_k \boldsymbol{u}_k$;
  - $\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k \boldsymbol{P}_{k-1} \boldsymbol{A}_k^T + \boldsymbol{Q}_k$;
  - **return** $\bar{\hat{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k$;
- **Method** `Update(`$\bar{\hat{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k, \boldsymbol{z}_k, \boldsymbol{R}_k, \boldsymbol{H}_k$`)`
  - $\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T (\boldsymbol{H}_k \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T + \boldsymbol{R}_k)^{-1}$;
  - $\hat{\boldsymbol{x}}_k = \bar{\hat{\boldsymbol{x}}}_k + \boldsymbol{K}_k (\boldsymbol{z}_k - \boldsymbol{H}_k \bar{\hat{\boldsymbol{x}}}_k)$;
  - $\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k) \bar{\boldsymbol{P}}_k (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k)^T + \boldsymbol{K}_k \boldsymbol{R}_k \boldsymbol{K}_k^T$;
  - **return** $\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k$;

Class *KF_1DOFAUV_CtVelocity_PositionOM : GFLocalization, KF*

- **Constructor** `KF_1DOFAUV_CtVelocity_PositionOM()`
  - $\boldsymbol{A_k} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$;
  - $\boldsymbol{B_k} = \boldsymbol{0}$;
  - $KF(\boldsymbol{A_k}, \boldsymbol{B_k})$; `// KF constructor`
- **Method** `GetInput()`
  - $\boldsymbol{u_k} = [\,]$; `// not used`
  - $\boldsymbol{Q_k} = \begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 \end{bmatrix}$; `// Motion Model Uncertainty`
  - **return** $[\boldsymbol{u_k}, \boldsymbol{Q_k}]$
- **Method** `GetMeasurements()`
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}] = GetDVL()$;
  - $\boldsymbol{H_k} = \begin{bmatrix} 0 & 1 \end{bmatrix}$;
  - **return** $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}]$

Summary

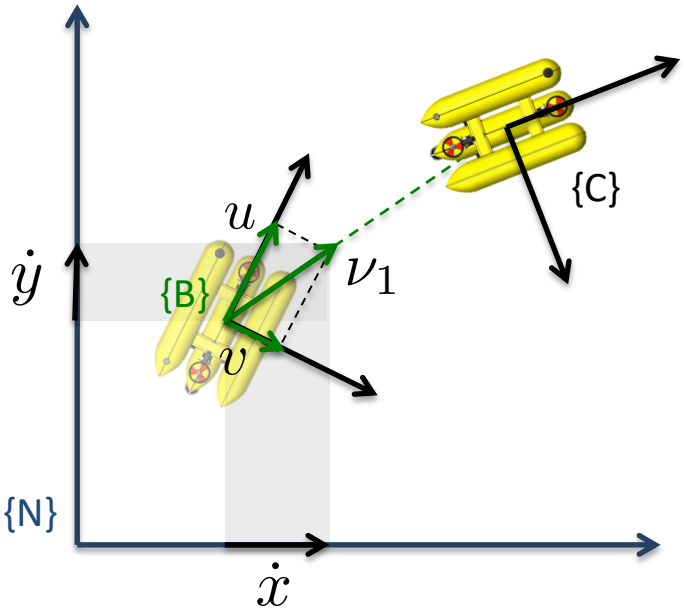
- **Highly efficient:** Polynomial in measurement dimensionality k and state dimensionality n:

  $$O(k^{2.376} + n^2)$$

- **Optimal** for linear Gaussian systems!

But...

- Most robotics systems are **nonlinear!**

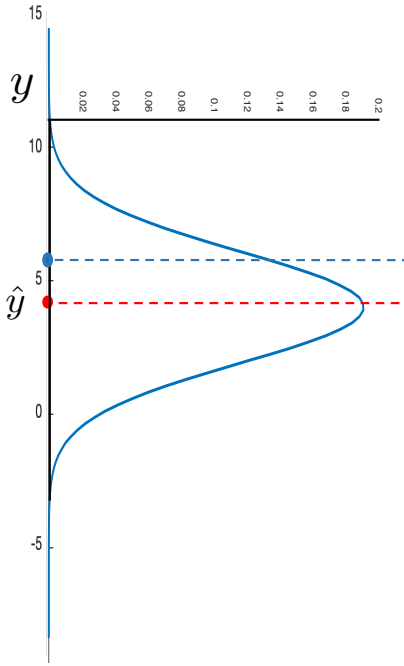


$$\bar{\boldsymbol{x}}_{\boldsymbol{k}} = \boldsymbol{x}_{\boldsymbol{k-1}} \oplus (\boldsymbol{u}_{\boldsymbol{k}} + \boldsymbol{w}_{\boldsymbol{k}})$$

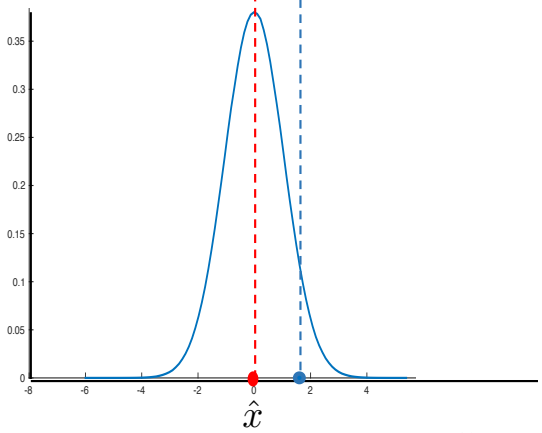
$$= \begin{bmatrix} {}^{N}x_B \\ {}^{N}y_B \\ {}^{N}z_B \\ {}^{N}\psi_B \end{bmatrix} \oplus \begin{bmatrix} ({}^{B}x_C + w_x) \\ ({}^{B}y_C + w_y) \\ ({}^{B}z_C + w_z) \\ ({}^{B}\psi_C + w_\psi) \end{bmatrix} = \begin{bmatrix} {}^{N}x_B + ({}^{B}x_C + w_x)\cos{}^{N}\psi_B - ({}^{B}y_C + w_y)\sin{}^{N}\psi_B \\ {}^{N}y_B + ({}^{B}x_C + w_x)\sin{}^{N}\psi_B + ({}^{B}y_C + w_y)\cos{}^{N}\psi_B \\ {}^{N}z_B + ({}^{B}z_C + w_z) \\ {}^{N}\psi_B + ({}^{B}\psi_C + w_\psi) \end{bmatrix}$$

where $\boldsymbol{u}_{\boldsymbol{k}} = \begin{bmatrix} {}^{B}x_C & {}^{B}y_C & {}^{B}z_C & {}^{B}\psi_C \end{bmatrix}^T$

Linearity Assumption Revised

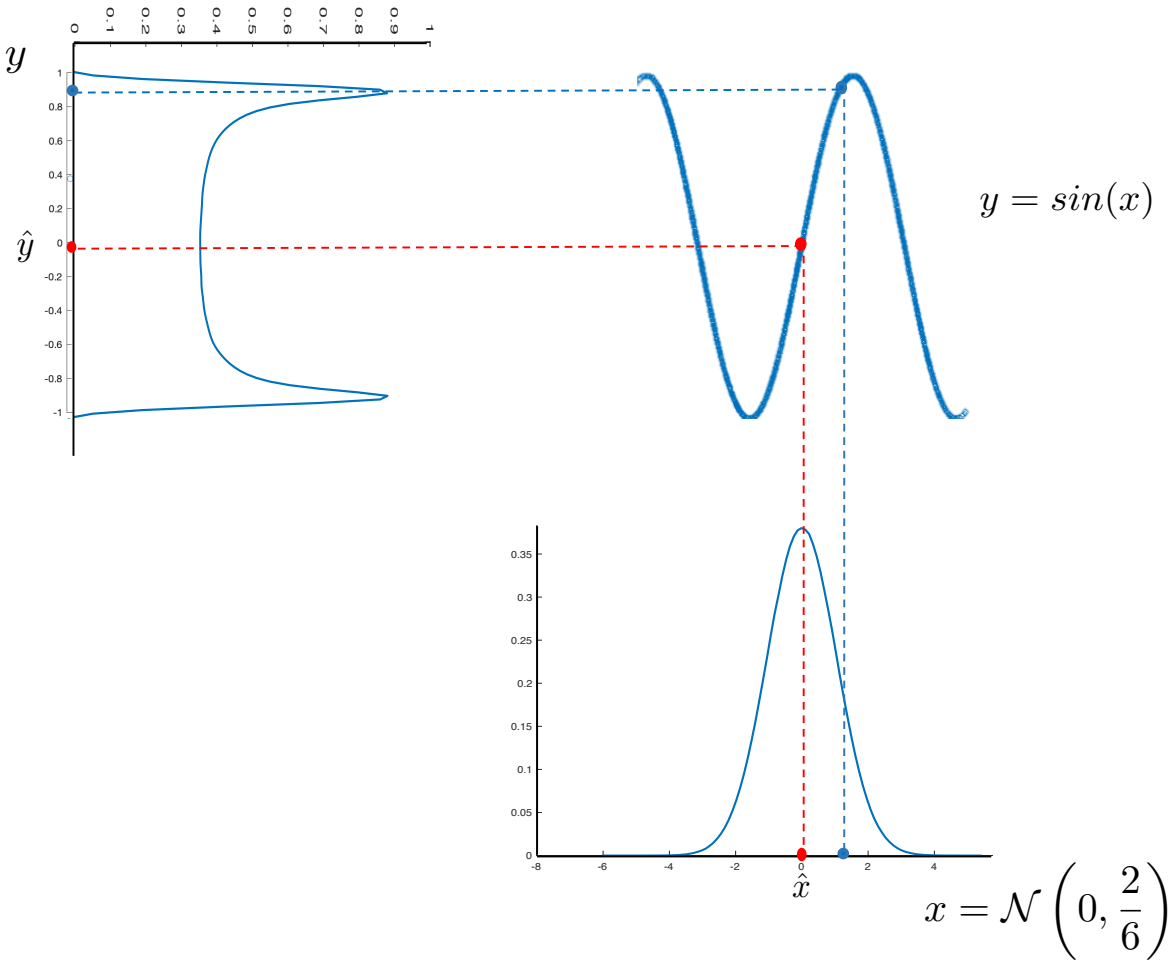


$y = 2x + 4$

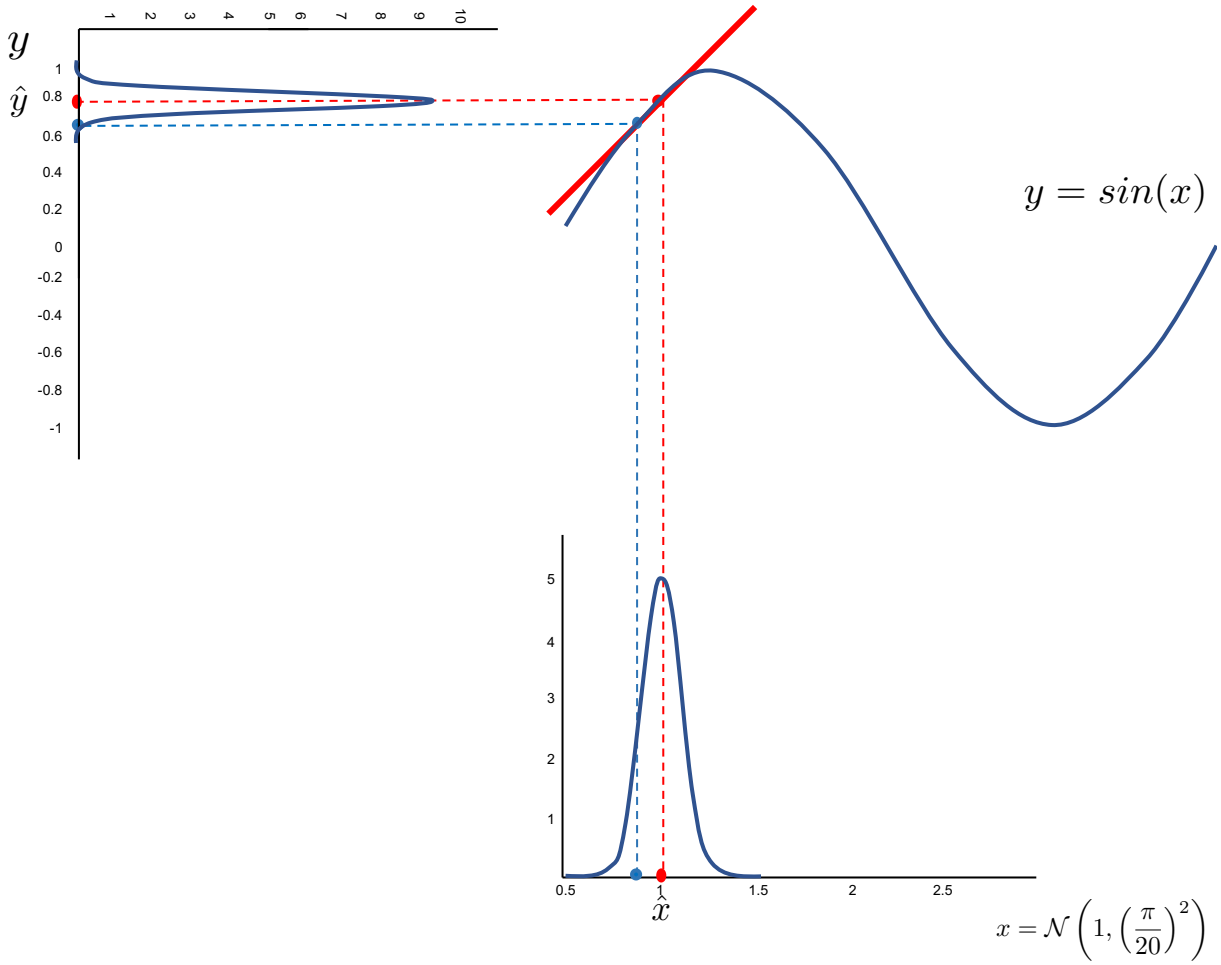


$$x = \mathcal{N}\left(0, \frac{2}{6}\right)$$

Non-linear Function



Non-linear Function



Gaussian Filters

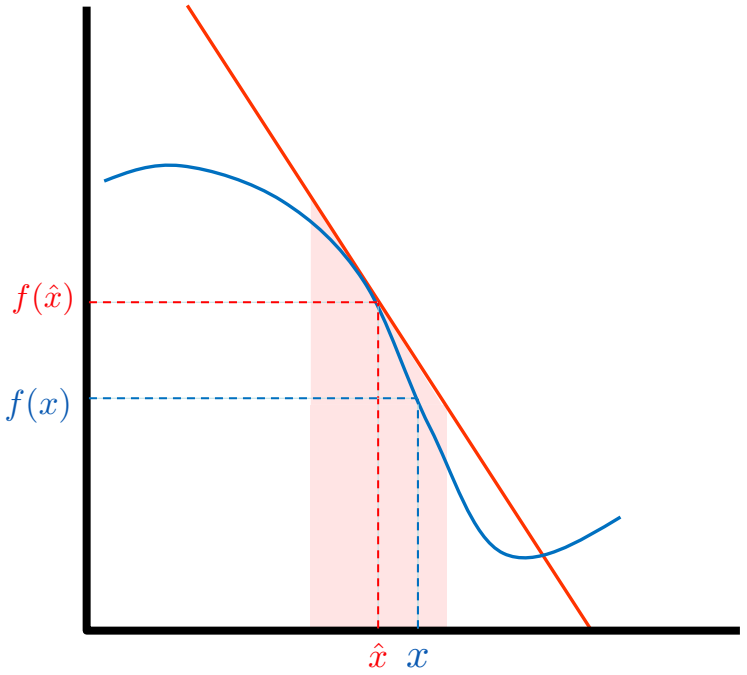
The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation



- We want to approximate $f(x)$ in an interval around $\hat{x}$

Gaussian Filters

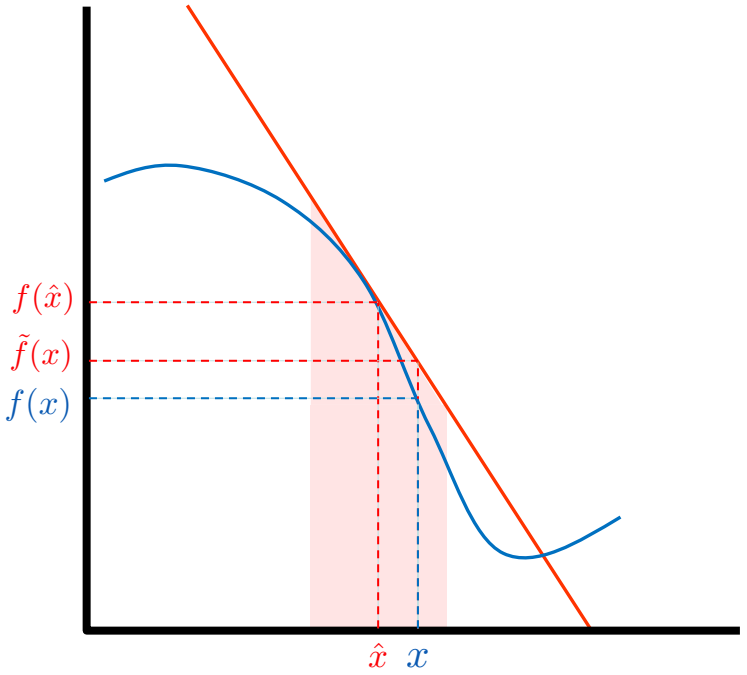
The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation



- We want to approximate $f(x)$ in an interval around $\hat{x}$

Gaussian Filters

The Extended Kalman Filter

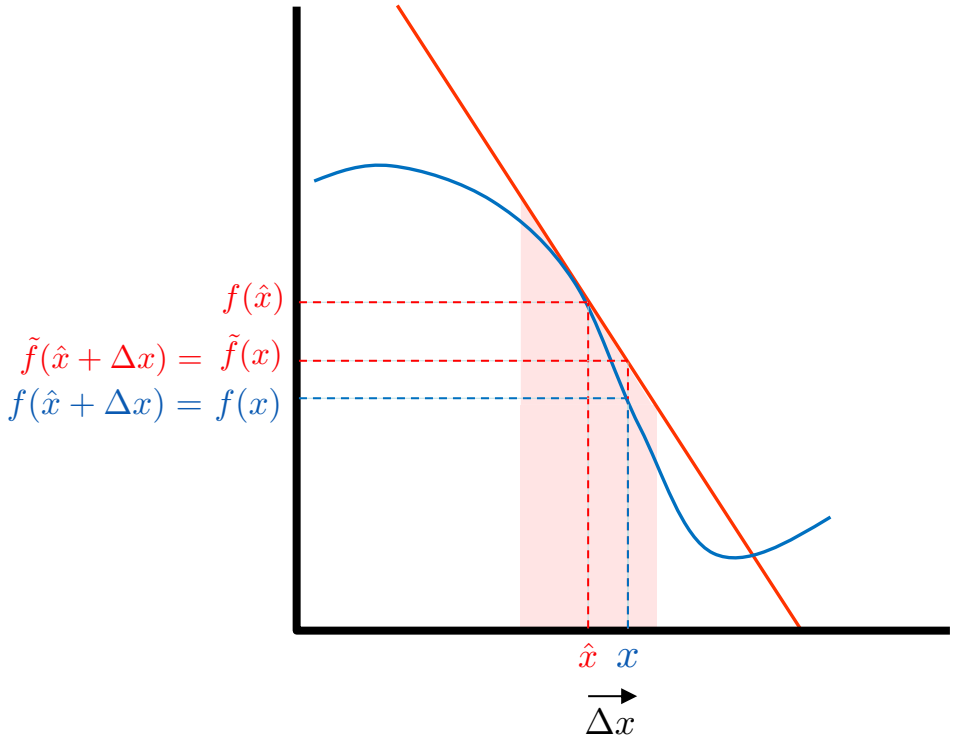
- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation

- *for a 1D function ...*



- We can express $x = \hat{x} + \Delta x$

Gaussian Filters

The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

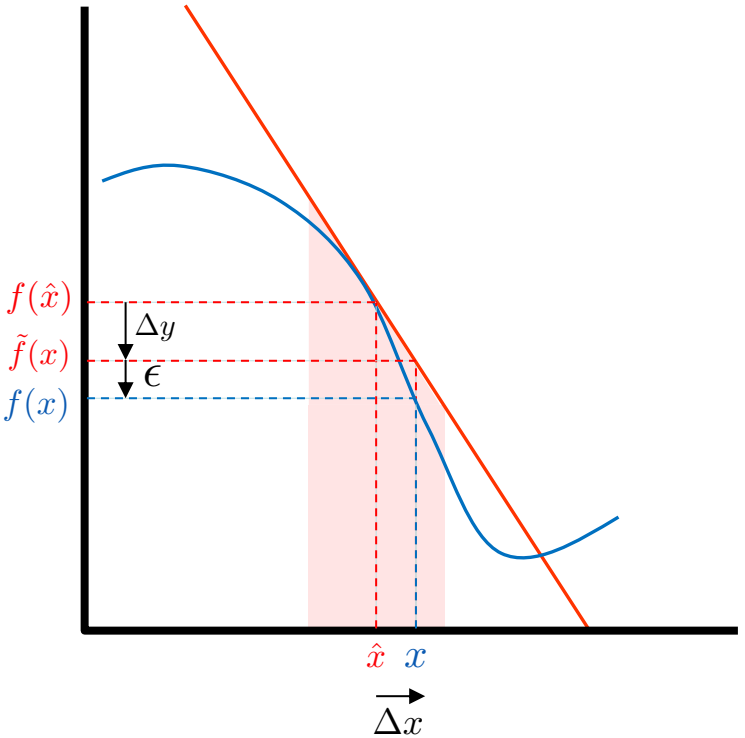
$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation

- *for a 1D function ...*

$$f(\hat{x}) + \left.\frac{df}{dx}\right|_{x=\hat{x}} (x - \hat{x}) = f(\hat{x}) + \frac{\Delta y}{\Delta x}\Delta x = \tilde{f}(\hat{x} + \Delta x) = \tilde{f}(x)$$
$$f(\hat{x} + \Delta x) = f(x)$$



- And $\tilde{f}(\hat{x} + \Delta x)$ can be expressed in terms of f derivative

Gaussian Filters

The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

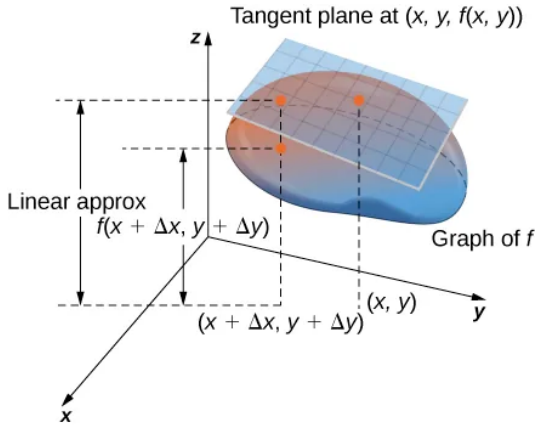
$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation

- *or for a 2D function ...*

$$\boldsymbol{f}(\boldsymbol{x}) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}) + \left.\frac{\partial \boldsymbol{f}(\boldsymbol{x})}{\partial \boldsymbol{x}}\right|_{\hat{\boldsymbol{x}}=\boldsymbol{x}} (\boldsymbol{x} - \hat{\boldsymbol{x}})$$

$$\boldsymbol{x} = [x \ y]^T$$
$$\hat{\boldsymbol{x}} = [\hat{x} \ \hat{y}]^T$$



$$\boldsymbol{f}\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} f_x(x,y) \\ f_y(x,y) \end{bmatrix} \approx \boldsymbol{f}\left(\begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix}\right) + \left.\begin{bmatrix} \frac{\partial \boldsymbol{f}_x(x,y)}{\partial x} & \frac{\partial \boldsymbol{f}_x(x,y)}{\partial y} \\ \frac{\partial \boldsymbol{f}_y(x,y)}{\partial x} & \frac{\partial \boldsymbol{f}_y(x,y)}{\partial y} \end{bmatrix}\right|_{[\boldsymbol{x} \ \boldsymbol{y}]^T = [\hat{\boldsymbol{x}} \ \hat{\boldsymbol{y}}]^T} \left(\begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix}\right)$$

Gaussian Filters

The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\Rightarrow$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation

Motion Model:

- *for the n D motion model, we linearize wrt the GRV x_k and w_k*

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \left[\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}} \quad \frac{\boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k} \right] \Big|_{\substack{\boldsymbol{x}_{k-1} = \hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k = \boldsymbol{0}}} \left(\begin{bmatrix} \boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k - \boldsymbol{0} \end{bmatrix} \right)$$

The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation

Motion Model:

- *for the n D motion model, we linearize wrt the GRV x_k and w_k*

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \left[\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}} \quad \frac{\boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k} \right] \Bigg|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}} \left(\begin{bmatrix} \boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k - \boldsymbol{0} \end{bmatrix} \right)$$

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \underbrace{\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}} \Bigg|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}}}_{\boldsymbol{A}_k} (\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k) + \underbrace{\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k} \Bigg|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}}}_{\boldsymbol{W}_k} \boldsymbol{w}_k$$

Gaussian Filters

The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

But it is possible to do a linear approximation

Motion Model:

- *for the n D motion model, we linearize wrt the GRV x_k and w_k*

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \left[\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}} \quad \frac{\boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k} \right] \Bigg|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}} \left(\begin{bmatrix} \boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k - \boldsymbol{0} \end{bmatrix} \right)$$

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \underbrace{\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}} \Bigg|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}}}_{\boldsymbol{A}_k} (\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k) + \underbrace{\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k} \Bigg|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}}}_{\boldsymbol{W}_k} \boldsymbol{w}_k$$

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \boldsymbol{A}_k \boldsymbol{x}_k - \boldsymbol{A}_k \hat{\boldsymbol{x}}_k + \boldsymbol{W}_k \boldsymbol{w}_k$$

Linear equation

Stochastic variables

Gaussian Filters

The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

EKF is a KF that makes use of a 1st order Linear Approximation

Motion Model:

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \underbrace{\left.\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}}\right|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}}}_{\boldsymbol{A}_k} (\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}) + \underbrace{\left.\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k}\right|_{\substack{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k=\boldsymbol{0}}}}_{\boldsymbol{W}_k} \boldsymbol{w}_k$$

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \boldsymbol{A}_k \boldsymbol{x}_k - \boldsymbol{A}_k \hat{\boldsymbol{x}}_k + \boldsymbol{W}_k \boldsymbol{w}_k$$

Linear equation

Stochastic variables

Gaussian Filters

The Extended Kalman Filter

- Most realistic robotic problems involve nonlinear functions

$$\mathbf{x}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k)$$
$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)$$

EKF is a KF that makes use of a 1st order Linear Approximation

Motion Model:

$$\bar{x}_k = f(x_{k-1}, u_k, w_k) \approx f(\hat{x}_{k-1}, u_k, 0) + \underbrace{\left.\frac{\partial f(x_{k-1}, u_k, w_k)}{\partial x_{k-1}}\right|_{\substack{x_{k-1}=\hat{x}_{k-1} \\ w_k=0}}}_{A_k} (x_{k-1} - \hat{x}_{k-1}) + \underbrace{\left.\frac{\partial f(x_{k-1}, u_k, w_k)}{\partial w_k}\right|_{\substack{x_{k-1}=\hat{x}_{k-1} \\ w_k=0}}}_{W_k} w_k$$

$$\bar{x}_k = f(x_{k-1}, u_k, w_k) \approx f(\hat{x}_{k-1}, u_k, 0) + A_k x_k - A_k \hat{x}_k + W_k w_k$$

Observation Model:

$$z_k = h(x_k, v_k) \approx h(\hat{x}_k, 0) + \underbrace{\left.\frac{\partial h(x_k, v_k)}{\partial x_k}\right|_{\substack{x_k=\hat{x}_k \\ v_k=0}}}_{H_k} (x_k - \hat{x}_k) + \underbrace{\left.\frac{\partial h(x_k, v_k)}{\partial v_k}\right|_{\substack{x_k=\hat{x}_k \\ v_k=0}}}_{V_k} v_k$$

Stochastic variables

$$z_k = h(x_k, v_k) \approx h(\hat{x}_k, 0) + H_k x_k - H_k \hat{x}_k + V_k v_k$$

Linear equation

Gaussian Filters

The Extended Kalman Filter

- The state is predicted using the nonlinear model

$$\hat{\bar{\boldsymbol{x}}}_k = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{B}_k \boldsymbol{u}_k + \boldsymbol{0} \quad \Rightarrow \quad \hat{\bar{\boldsymbol{x}}}_k = \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0});$$

- The covariance is projected using the linear approximation

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0}) + \boldsymbol{A_k x_{k-1}} - \boldsymbol{A_k \hat{x}_{k-1}} + \boldsymbol{W_k w_k}$$

$$\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k \boldsymbol{P}_{k-1} \boldsymbol{A}_k^T + \boldsymbol{W}_k \boldsymbol{Q}_k \boldsymbol{W}_k^T;$$

$$\boldsymbol{A_k} = \left. \frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}} \right|_{\substack{\boldsymbol{x}_{k-1} = \hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k = \boldsymbol{0}}}$$

$$\boldsymbol{W_k} = \left. \frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k} \right|_{\substack{\boldsymbol{x}_{k-1} = \hat{\boldsymbol{x}}_{k-1} \\ \boldsymbol{w}_k = \boldsymbol{0}}}$$

Kalman_Filter$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)$

$\hat{\bar{\boldsymbol{x}}}_k = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{B}_k \boldsymbol{u}_k;$

$\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k \boldsymbol{P}_{k-1} \boldsymbol{A}_k^T + \boldsymbol{Q}_k;$

$\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T (\boldsymbol{H}_k \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T + \boldsymbol{R}_k)^{-1};$

$\hat{\boldsymbol{x}}_{\boldsymbol{k}} = \hat{\bar{\boldsymbol{x}}}_k + \boldsymbol{K}_k (\boldsymbol{z}_k - \boldsymbol{H}_k \hat{\bar{\boldsymbol{x}}}_k);$

$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k) \bar{\boldsymbol{P}}_k;$

Return$([\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P}_{\boldsymbol{k}}]);$

Extended_Kalman_Filter$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)$

$\hat{\bar{\boldsymbol{x}}}_k = \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0});$

$\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k \boldsymbol{P}_{k-1} \boldsymbol{A}_k^T + \boldsymbol{W}_k \boldsymbol{Q}_k \boldsymbol{W}_k^T;$

$\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T (\boldsymbol{H}_k \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T + \boldsymbol{V}_k \boldsymbol{R}_k \boldsymbol{V}_k^T)^{-1};$

$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{K}_k (\boldsymbol{z}_k - h(\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}}, 0));$

$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k) \bar{\boldsymbol{P}}_k;$

Return$([\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P}_{\boldsymbol{k}}]);$

The Extended Kalman Filter

- Since the measurement noise is nonlinear ….

$$\boldsymbol{z_k = H_k x_k + v_k} \quad \Rightarrow \quad \boldsymbol{z_k = h(x_k, v_k)}$$

- Its covariance has to be linearized within the K gain

$$\boldsymbol{K_k = \bar{P}_k H_k^T (H_k \bar{P}_k H_k^T + V_k R_k V_k^T)^{-1};}$$

$$\boldsymbol{H_k = \left.\frac{\partial h(x_k, v_k)}{\partial x_k}\right|_{x_k = \hat{x}_k, \hat{v}_k = 0}; \quad V_k = \left.\frac{\partial h(x_k, v_k)}{\partial v_k}\right|_{x_k = \hat{x}_k, \hat{v}_k = 0}}$$

Kalman_Filter$(\hat{x}_{k-1}, u_k, w_k)$

$\hat{\bar{x}}_k = A_k \hat{x}_{k-1} + B_k u_k;$

$\bar{P}_k = A_k P_{k-1} A_k^T + Q_k;$

$K_k = \bar{P}_k H_k^T (H_k \bar{P}_k H_k^T + R_k)^{-1};$

$\hat{x}_k = \hat{\bar{x}}_k + K_k (z_k - H_k \hat{\bar{x}}_k);$

$P_k = (I - K_k H_k) \bar{P}_k;$

Return$([\hat{x}_k, P_k]);$

Extended_Kalman_Filter$(\hat{x}_{k-1}, u_k, w_k)$

$\hat{\bar{x}}_k = f(\hat{x}_{k-1}, u_k, 0);$

$\bar{P}_k = A_k P_{k-1} A_k^T + W_k Q_k W_k^T;$

$K_k = \bar{P}_k H_k^T (H_k \bar{P}_k H_k^T + V_k R_k V_k^T)^{-1};$

$\hat{x}_k = \hat{x}_{k-1} + K_k (z_k - h(\hat{\bar{x}}_k, 0));$

$P_k = (I - K_k H_k) \bar{P}_k;$

Return$([\hat{x}_k, P_k]);$

The Extended Kalman Filter

- Since the measurement noise is nonlinear ….

$$\boldsymbol{z_k = H_k x_k + v_k} \quad \Rightarrow \quad \boldsymbol{z_k = h(x_k, v_k)}$$

- Update equation remains almost the same

$$\hat{\boldsymbol{x}}_{\boldsymbol{k}} = \bar{\hat{\boldsymbol{x}}}_k + \boldsymbol{K}_k(\boldsymbol{z}_k - \boldsymbol{H}_k\bar{\hat{\boldsymbol{x}}}_k); \quad \Rightarrow \quad \hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{K}_k(\boldsymbol{z}_k - h(\bar{\hat{\boldsymbol{x}}}_{\boldsymbol{k}}, 0));$$
$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H}_k)\bar{\boldsymbol{P}}_k; \qquad \boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H}_k)\bar{\boldsymbol{P}}_k;$$

Kalman_Filter$(\hat{x}_{k-1}, u_k, w_k)$

$$\bar{\hat{\boldsymbol{x}}}_k = \boldsymbol{A}_k\hat{\boldsymbol{x}}_{k-1} + \boldsymbol{B}_k\boldsymbol{u}_k;$$
$$\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k\boldsymbol{P}_{k-1}\boldsymbol{A}_k^T + \boldsymbol{Q}_k;$$
$$\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k\boldsymbol{H}_k^T(\boldsymbol{H}_k\bar{\boldsymbol{P}}_k\boldsymbol{H}_k^T + \boldsymbol{R}_k)^{-1};$$
$$\hat{\boldsymbol{x}}_{\boldsymbol{k}} = \bar{\hat{\boldsymbol{x}}}_k + \boldsymbol{K}_k(\boldsymbol{z}_k - \boldsymbol{H}_k\bar{\hat{\boldsymbol{x}}}_k);$$
$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H}_k)\bar{\boldsymbol{P}}_k;$$

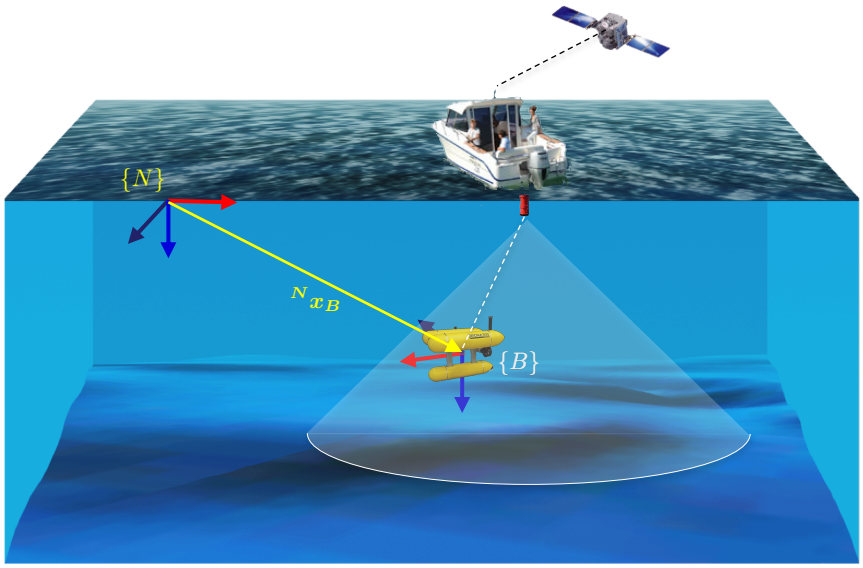
Return$([\hat{x}_k, P_k])$;

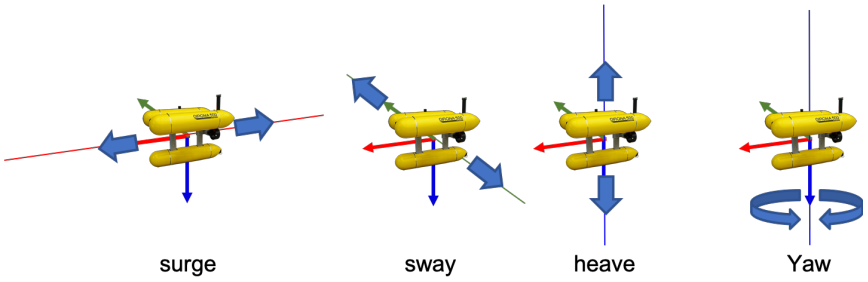
Extended_Kalman_Filter$(\hat{x}_{k-1}, u_k, w_k)$

$$\bar{\hat{\boldsymbol{x}}}_k = \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k, \boldsymbol{0});$$
$$\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k\boldsymbol{P}_{k-1}\boldsymbol{A}_k^T + \boldsymbol{W}_k\boldsymbol{Q}_k\boldsymbol{W}_k^T;$$
$$\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k\boldsymbol{H}_k^T(\boldsymbol{H}_k\bar{\boldsymbol{P}}_k\boldsymbol{H}_k^T + \boldsymbol{V}_k\boldsymbol{R}_k\boldsymbol{V}_k^T)^{-1};$$
$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{K}_k(\boldsymbol{z}_k - h(\bar{\hat{\boldsymbol{x}}}_{\boldsymbol{k}}, 0));$$
$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H}_k)\bar{\boldsymbol{P}}_k;$$

Return$([\hat{x}_k, P_k])$;

Example: AUV EKF Localization with Displacement and USBL fixes





4 DOF AUV EKF

> Define the **State Vector:**

$$\boldsymbol{x_k} = {}^{\boldsymbol{N}}\,\boldsymbol{x_B} = [{}^{N}x_B \quad {}^{N}y_B \quad {}^{N}z_B \quad {}^{N}\psi_B]^T$$

> Define the **Motion Model** with an explicit noise representation

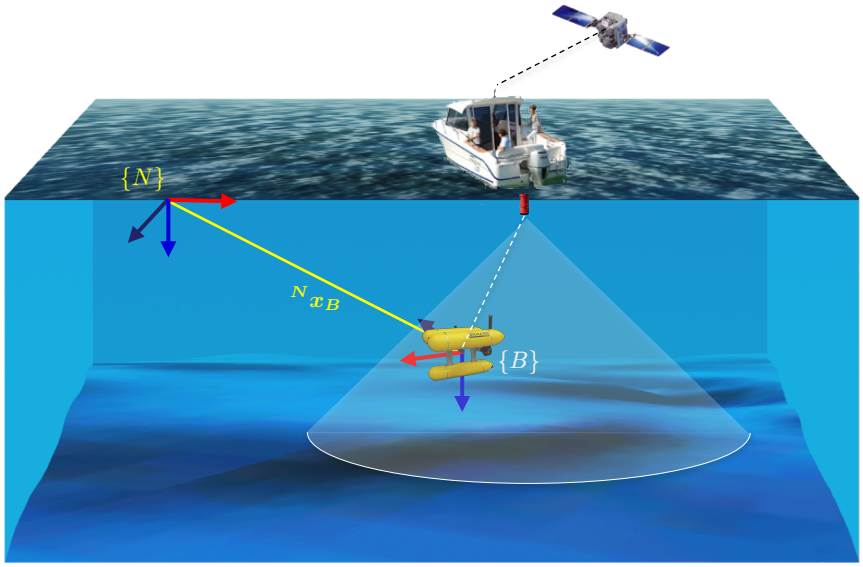
- AUV displacement measured with a DVL and a 1 axis gyroscope

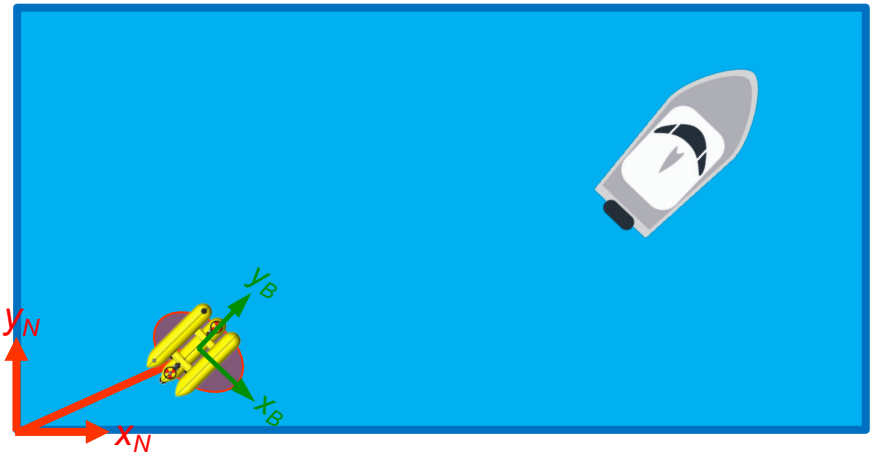
> Define the **Observation model** with an explicit noise representation

- 3D DGPS-AHRS-USBL position
- Depth sensor
- Compass sensor

Gaussian Filters

Example: AUV EKF Localization with Displacement and USBL fixes



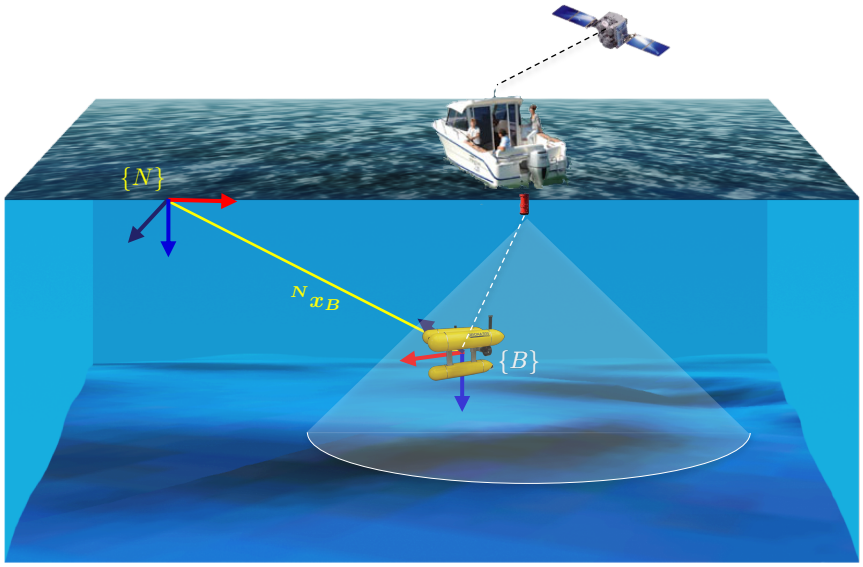


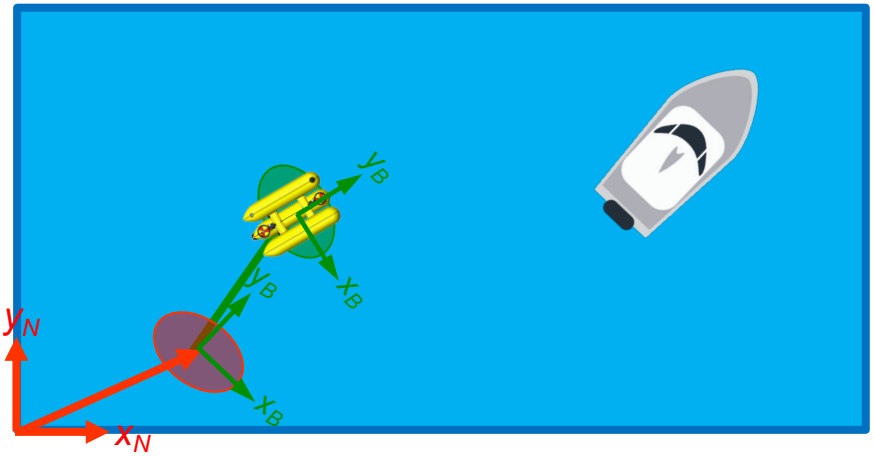
Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop`$(\hat{\boldsymbol{x}}_0, \boldsymbol{P_0})$
  - $[\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}] = [\hat{\boldsymbol{x}}_0, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_k, \boldsymbol{P_k}] = Localize(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
- **Method** `Localize`$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\hat{\bar{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k] = Prediction(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_k, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_k, \boldsymbol{P_k}]$

Gaussian Filters

Example: AUV EKF Localization with Displacement and USBL fixes



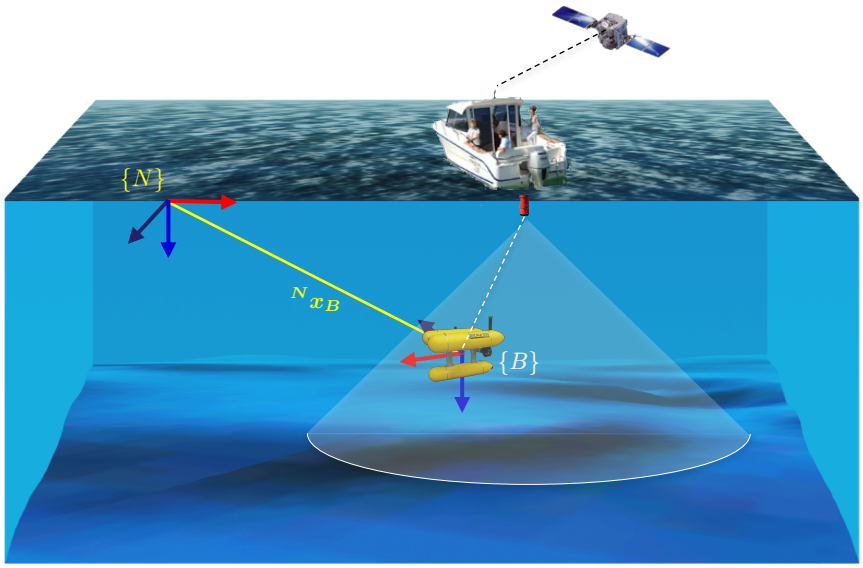


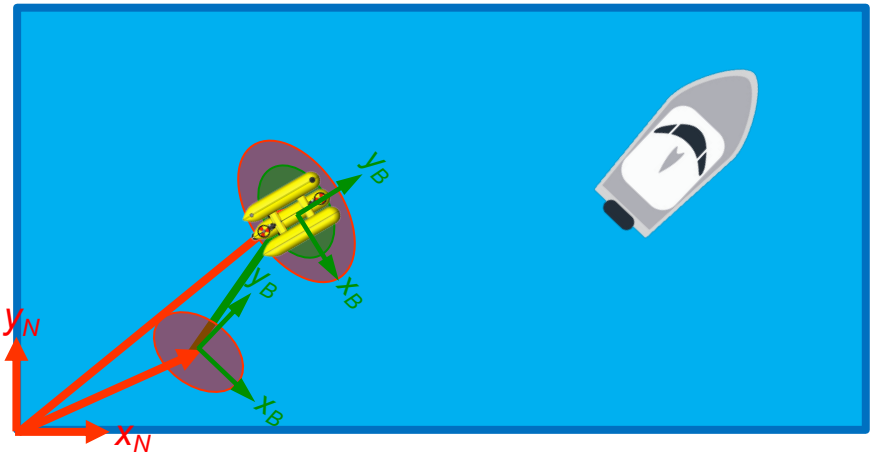
Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop`$(\hat{\boldsymbol{x}}_0, \boldsymbol{P_0})$
  - $[\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}] = [\hat{\boldsymbol{x}}_0, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k] = Localize(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
- **Method** `Localize`$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\hat{\bar{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k] = Prediction(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_k, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k]$

Gaussian Filters

Example: AUV EKF Localization with Displacement and USBL fixes



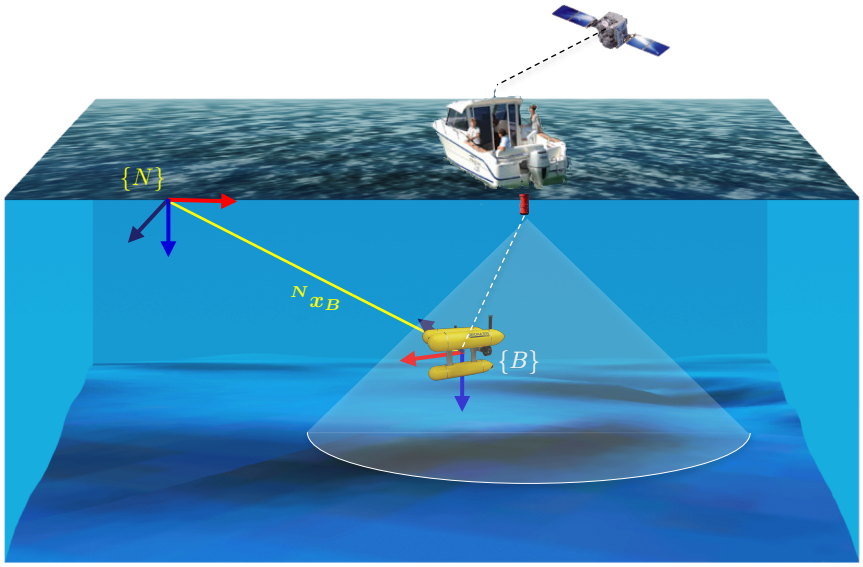


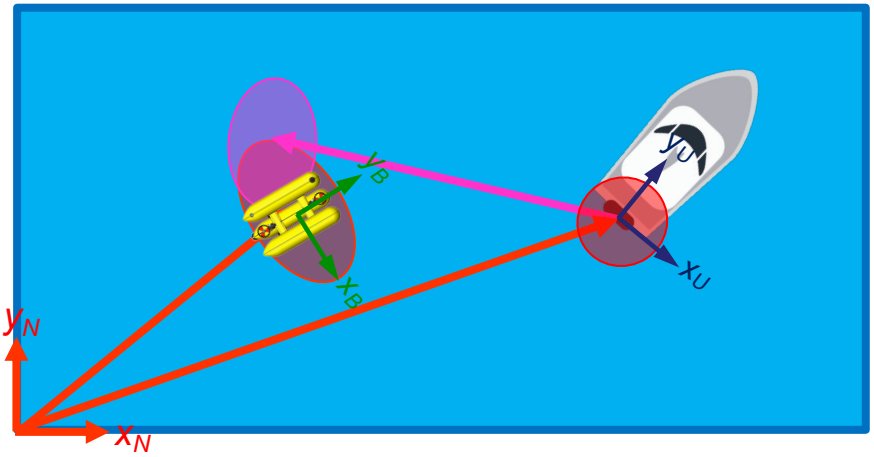
Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop`$(\hat{\boldsymbol{x}}_{\boldsymbol{0}}, \boldsymbol{P_0})$
  - $[\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}] = [\hat{\boldsymbol{x}}_{\boldsymbol{0}}, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k}] = Localize(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}})$
- **Method** `Localize`$(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}})$
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\bar{\hat{\boldsymbol{x}}}_{\boldsymbol{k}}, \bar{\boldsymbol{P}}_{\boldsymbol{k}}] = Prediction(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k}]$

Gaussian Filters

Example: AUV EKF Localization with Displacement and USBL fixes



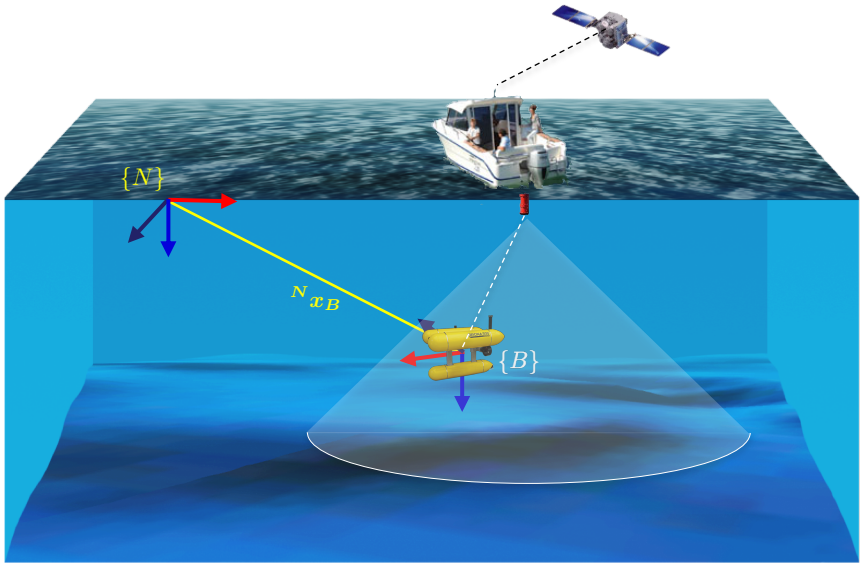


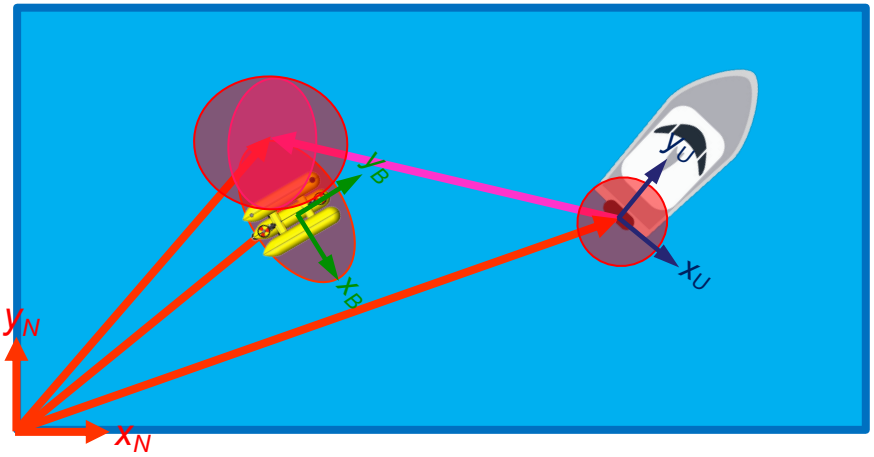
Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop`$(\hat{\boldsymbol{x}}_0, \boldsymbol{P_0})$
  - $[\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}] = [\hat{\boldsymbol{x}}_0, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k] = Localize(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
- **Method** `Localize`$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\hat{\bar{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k] = Prediction(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_k, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k]$

Gaussian Filters

Example: AUV EKF Localization with Displacement and USBL fixes



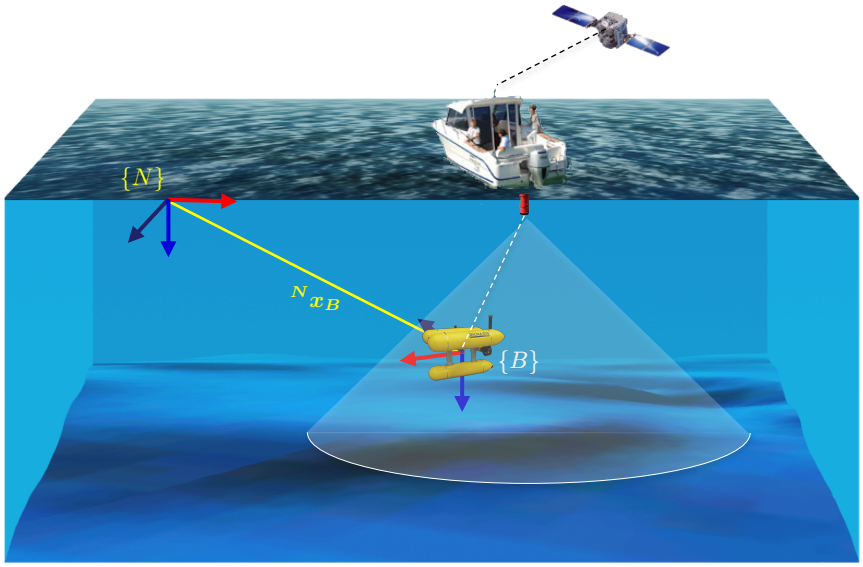


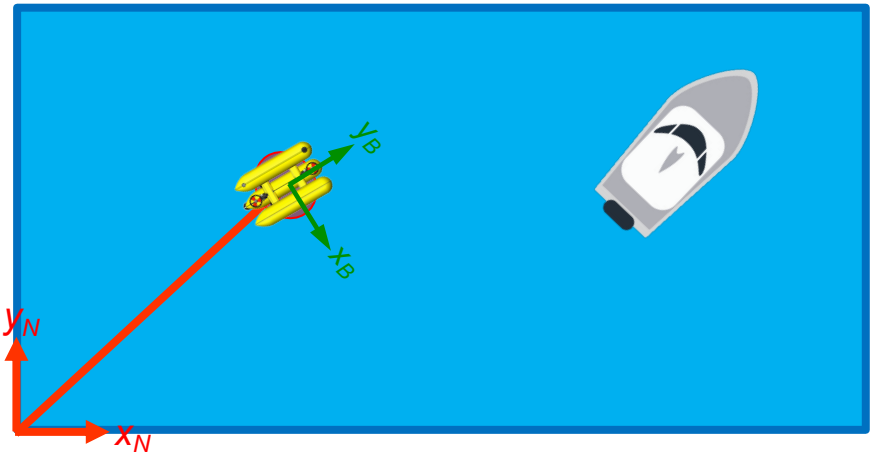
Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop`$(\hat{\boldsymbol{x}}_0, \boldsymbol{P_0})$
  - $[\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}] = [\hat{\boldsymbol{x}}_0, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k] = Localize(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
- **Method** `Localize`$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\hat{\bar{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k] = Prediction(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_k, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_k, \boldsymbol{P_k}]$

Gaussian Filters

Example: AUV EKF Localization with Displacement and USBL fixes

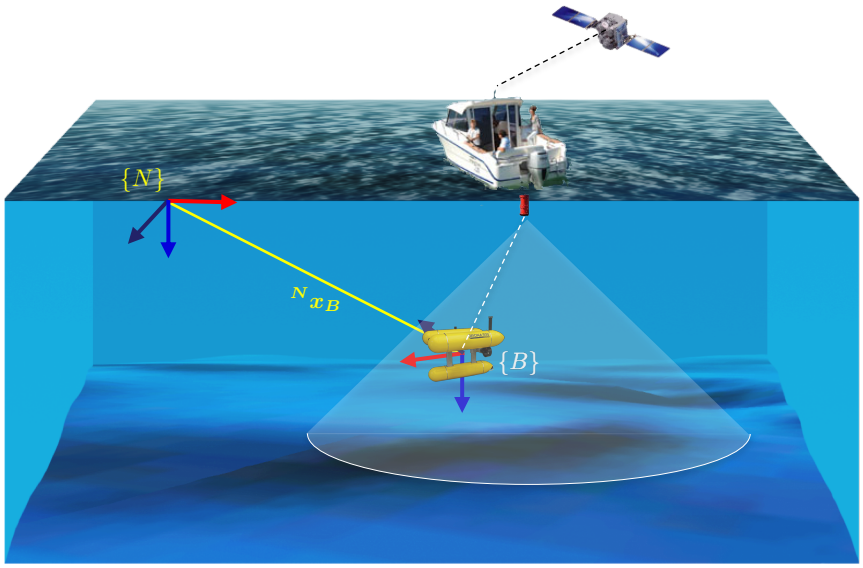


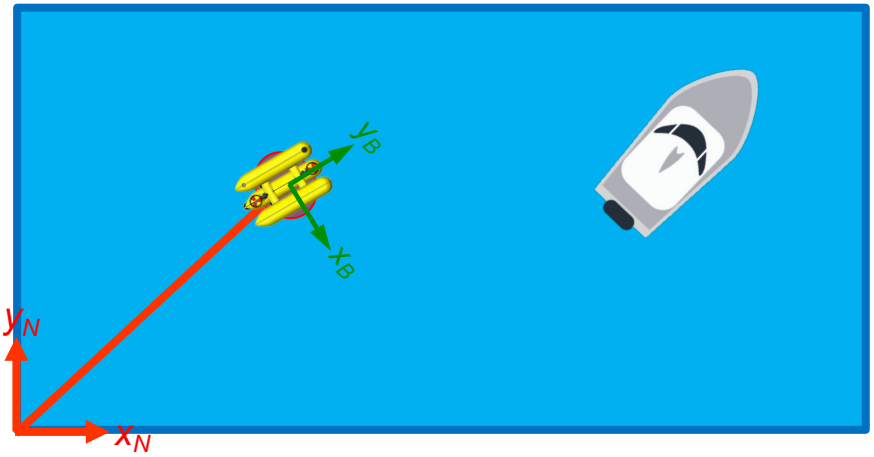


Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop(`$\hat{\boldsymbol{x}}_{\boldsymbol{0}}, \boldsymbol{P_0}$`)`
  - $[\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}] = [\hat{\boldsymbol{x}}_{\boldsymbol{0}}, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k}] = \mathit{Localize}(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}})$
- **Method** `Localize(`$\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}$`)`
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = \mathit{GetInput}()$;
  - $[\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}}, \bar{\boldsymbol{P}}_{\boldsymbol{k}}] = \mathit{Prediction}(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = \mathit{GetMeasurements}()$;
  - $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \bar{P}_k] = \mathit{Update}(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}, \boldsymbol{P_{k-1}}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k}]$

Example: AUV EKF Localization with Displacement and USBL fixes





Motion Model

- Vector State:

$$\boldsymbol{x_k} = \mathcal{N}(\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k})$$
$$\boldsymbol{x_k} = \begin{bmatrix} {}^{N}x_B & {}^{N}y_B & {}^{N}z_B & {}^{N}\psi_B \end{bmatrix}^T$$

- Stochastic motion model:

$$\bar{\boldsymbol{x}}_{\boldsymbol{k}} = \boldsymbol{f}(\boldsymbol{x_{k-1}}, \boldsymbol{u_k}, \boldsymbol{w_k}) = \boldsymbol{x_{k-1}} \oplus (\boldsymbol{u_k} + \boldsymbol{w_k})$$

where $\boldsymbol{x_k} = {}^{\boldsymbol{N}}\,\boldsymbol{x}_{\boldsymbol{B_k}}, \boldsymbol{x_{k-1}} = {}^{\boldsymbol{N}}\,\boldsymbol{x}_{\boldsymbol{B_{k-1}}}$ and $\boldsymbol{u_k} = {}^{\boldsymbol{B}}\,\boldsymbol{\Delta x_k}$.

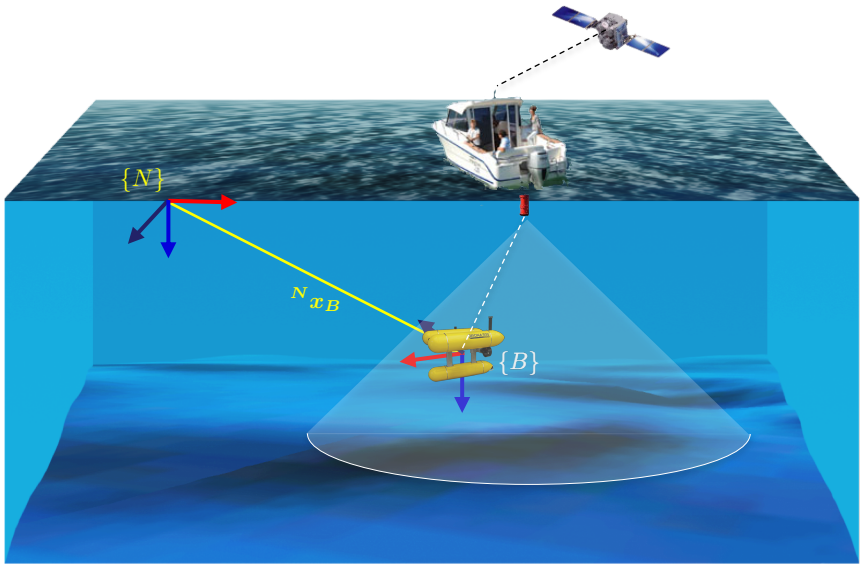
- Linear approximation:

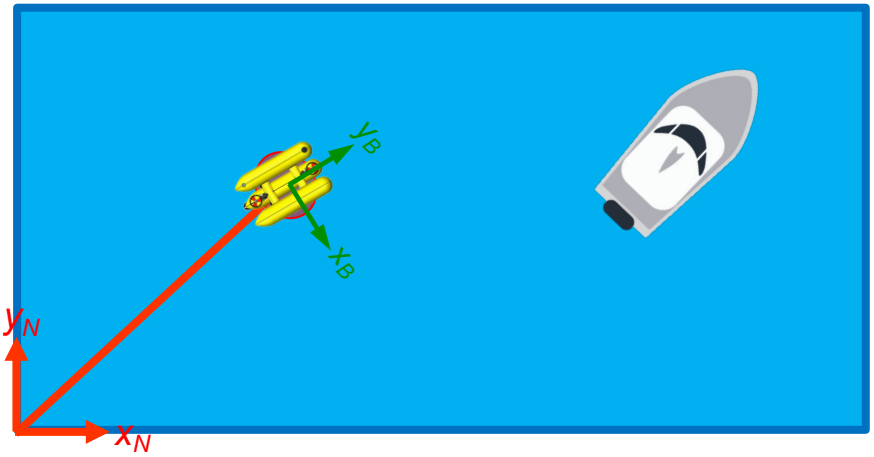
$$\boldsymbol{x_k} \approx \hat{\boldsymbol{x}}_{\boldsymbol{k-1}} \oplus \boldsymbol{u_k} + \boldsymbol{J}_{1\oplus}(\boldsymbol{x_{k-1}} - \hat{\boldsymbol{x}}_{\boldsymbol{k-1}}) + \boldsymbol{J}_{2\oplus}\boldsymbol{w_k}$$

where:

$$\boldsymbol{J}_{1\oplus} = \left.\frac{\partial \boldsymbol{x_{k-1}} \oplus (\boldsymbol{u_k} + \boldsymbol{w_k})}{\partial \boldsymbol{x_{k-1}}}\right|_{\boldsymbol{x_k}=\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{w_k}=\boldsymbol{0}}$$
$$\boldsymbol{J}_{2\oplus} = \left.\frac{\partial \boldsymbol{x_{k-1}} \oplus (\boldsymbol{u_k} + \boldsymbol{w_k})}{\partial \boldsymbol{w_k}}\right|_{\boldsymbol{x_k}=\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{w_k}=\boldsymbol{0}}$$

Example: AUV EKF Localization with Displacement and USBL fixes





Motion Model

- Vector State:

$$\boldsymbol{x_k} = \mathcal{N}(\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P_k})$$
$$\boldsymbol{x_k} = \begin{bmatrix} {}^N x_B & {}^N y_B & {}^N z_B & {}^N \psi_B \end{bmatrix}^T$$

- Stochastic motion model:

$$\bar{\boldsymbol{x}}_{\boldsymbol{k}} = \boldsymbol{f}(\boldsymbol{x_{k-1}}, \boldsymbol{u_k}, \boldsymbol{w_k}) = \boldsymbol{x_{k-1}} \oplus (\boldsymbol{u_k} + \boldsymbol{w_k})$$

where $\boldsymbol{x_k} = {}^{\boldsymbol{N}}\, \boldsymbol{x_{B_k}}, \boldsymbol{x_{k-1}} = {}^{\boldsymbol{N}}\, \boldsymbol{x_{B_{k-1}}}$ and $\boldsymbol{u_k} = {}^{\boldsymbol{B}}\, \boldsymbol{\Delta x_k}$.

- Linear approximation:

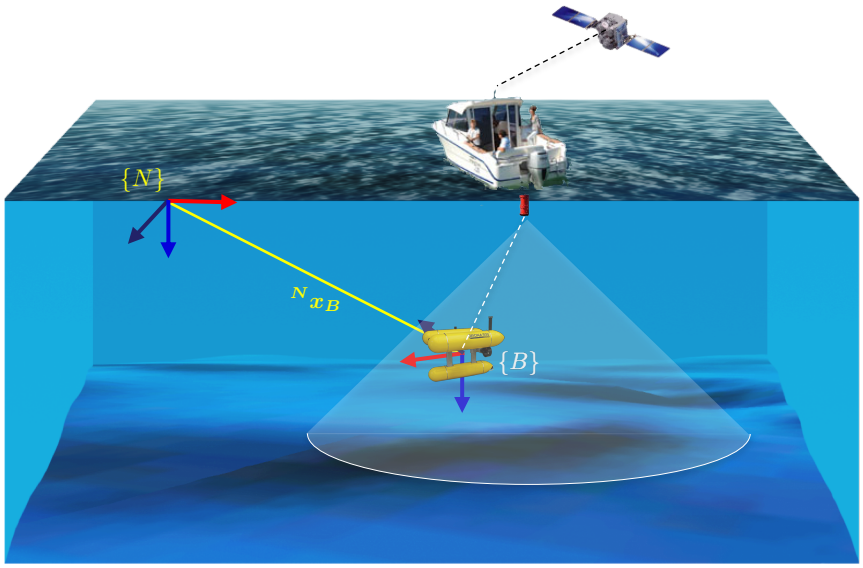
$$\boldsymbol{x_k} \approx \hat{\boldsymbol{x}}_{\boldsymbol{k-1}} \oplus \boldsymbol{u_k} + \boldsymbol{J_{1\oplus}}(\boldsymbol{x_{k-1}} - \hat{\boldsymbol{x}}_{\boldsymbol{k-1}}) + \boldsymbol{J_{2\oplus}} \boldsymbol{w_k}$$

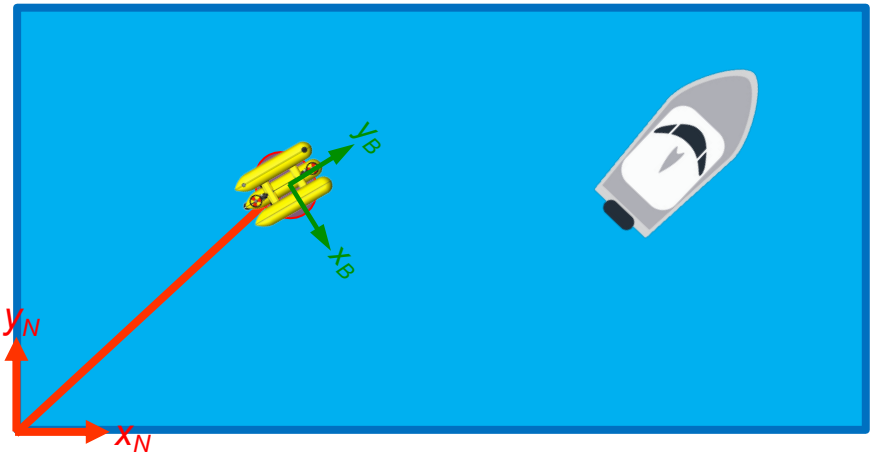
- The pose mean and covariance is given by:

$$\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}} = \hat{\boldsymbol{x}}_{\boldsymbol{k-1}} \oplus \boldsymbol{u_k}$$
$$\bar{\boldsymbol{P}}_{\boldsymbol{k}} = \boldsymbol{J_{1\oplus}} \boldsymbol{P}_{\boldsymbol{k-1}} \boldsymbol{J}_{\boldsymbol{1\oplus}}^{\boldsymbol{T}} + \boldsymbol{J_{2\oplus}} \boldsymbol{Q_k} \boldsymbol{J}_{\boldsymbol{2\oplus}}^{\boldsymbol{T}}$$

How are they computed?

Example: AUV EKF Localization with Displacement and USBL fixes





> Given the **motion model**

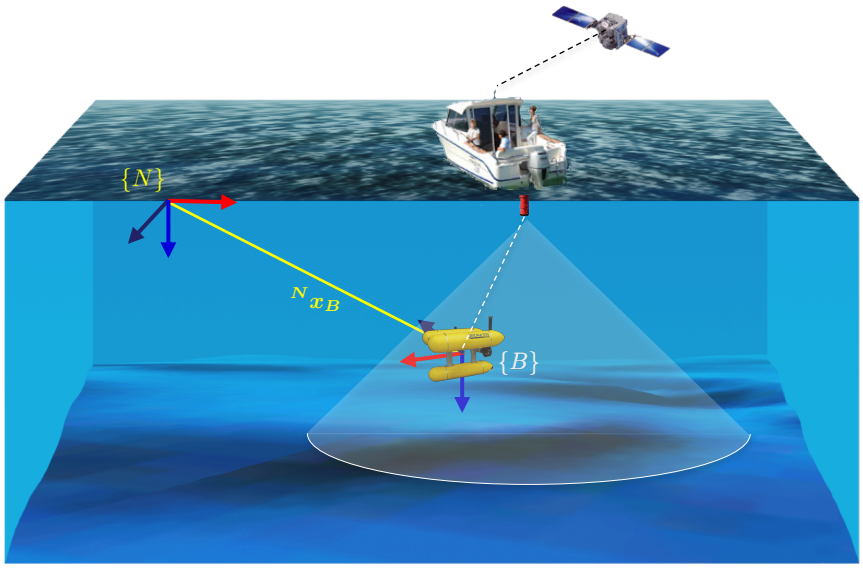
$$\bar{\boldsymbol{x}}_{\boldsymbol{k}} = \boldsymbol{f}(\boldsymbol{x}_{\boldsymbol{k}}, \boldsymbol{u}_{\boldsymbol{k}}, \boldsymbol{w}_{\boldsymbol{k}}) = \boldsymbol{x}_{\boldsymbol{k-1}} \oplus (\boldsymbol{u}_{\boldsymbol{k}} + \boldsymbol{w}_{\boldsymbol{k}})$$

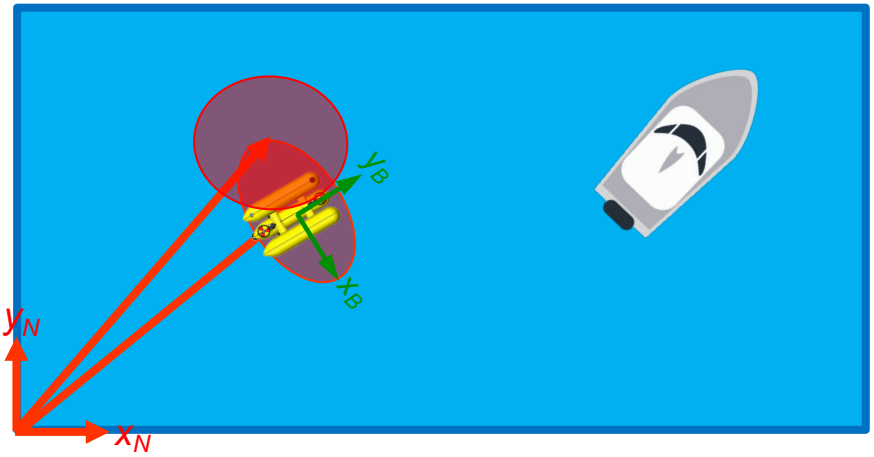
$$= \begin{bmatrix} {}^N x_B \\ {}^N y_B \\ {}^N z_B \\ {}^N \psi_B \end{bmatrix} \oplus \begin{bmatrix} ({}^B x_C + w_x) \\ ({}^B y_C + w_y) \\ ({}^B z_C + w_z) \\ ({}^B \psi_C + w_\psi) \end{bmatrix}$$

$$= \begin{bmatrix} {}^N x_B + ({}^B x_C + w_x)\cos {}^N\psi_B - ({}^B y_C + w_y)\sin {}^N\psi_B \\ {}^N y_B + ({}^B x_C + w_x)\sin {}^N\psi_B + ({}^B y_C + w_y)\cos {}^N\psi_B \\ {}^N z_B + ({}^B z_C + w_z) \\ {}^N \psi_B + ({}^B \psi_C + w_\psi) \end{bmatrix}$$

where $\boldsymbol{u}_{\boldsymbol{k}} = \begin{bmatrix} {}^B x_C & {}^B y_C & {}^B z_C & {}^B \psi_C \end{bmatrix}^T$

Example: AUV EKF Localization with Displacement and USBL fixes





> Given the **motion model**

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)$$

$$= \begin{bmatrix} {}^N x_B + ({}^B x_C + w_x)\cos{}^N\psi_B - ({}^B y_C + w_y)\sin{}^N\psi_B \\ {}^N y_B + ({}^B x_C + w_x)\sin{}^N\psi_B + ({}^B y_C + w_y)\cos{}^N\psi_B \\ {}^N z_B + ({}^B z_C + w_z) \\ {}^N\psi_B + ({}^B\psi_C + w_\psi) \end{bmatrix}.$$

> The **Jacobians** can be computed

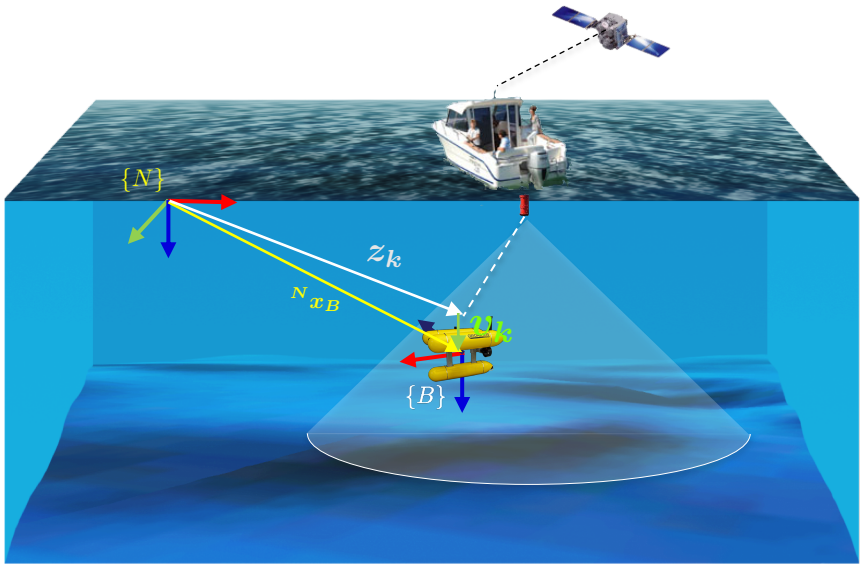
$$\boldsymbol{J}_{1\oplus} = \left.\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_k}\right|_{\boldsymbol{x}_k = \hat{\boldsymbol{x}}_k, \boldsymbol{w}_k = \boldsymbol{0}}$$

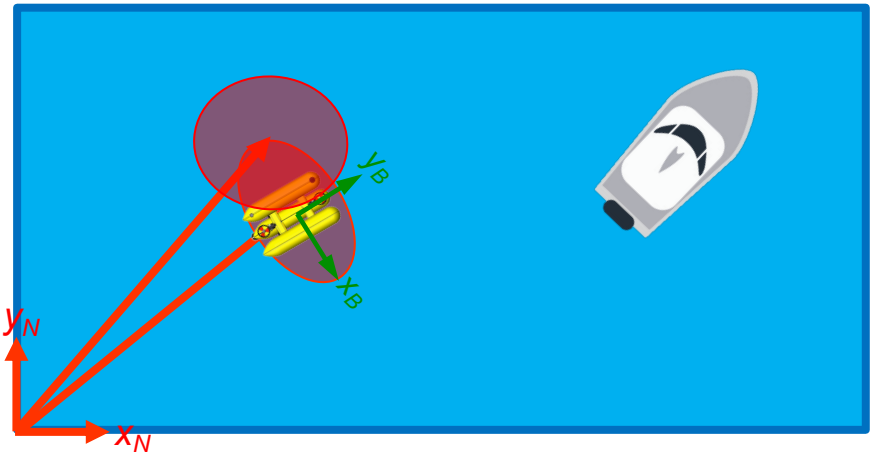
$$= \begin{bmatrix} 1 & 0 & 0 & -{}^B\hat{x}_C \cdot \sin{}^N\hat{\psi}_B - {}^B\hat{y}_C \cdot \cos{}^N\hat{\psi}_B \\ 0 & 1 & 0 & {}^B\hat{x}_C \cdot \cos{}^N\hat{\psi}_B - {}^B\hat{y}_C \cdot \sin{}^N\hat{\psi}_B \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\boldsymbol{J}_{2\oplus} = \left.\frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k}\right|_{\boldsymbol{x}_k = \hat{\boldsymbol{x}}_k, \hat{\boldsymbol{w}}_k = \boldsymbol{0}}$$

$$= \begin{bmatrix} \cos{}^N\hat{\psi}_B & -\sin{}^N\hat{\psi}_B & 0 & 0 \\ \sin{}^N\hat{\psi}_B & \cos{}^N\hat{\psi}_B & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Example: AUV EKF Localization with Displacement and USBL fixes





> The **Observation equation** is given by:

$$\boldsymbol{z_k} = \boldsymbol{h}(\boldsymbol{x_k}, \boldsymbol{v_k}) \; ; \; \boldsymbol{v_k} = \mathcal{N}(\boldsymbol{0}, \boldsymbol{R_k})$$

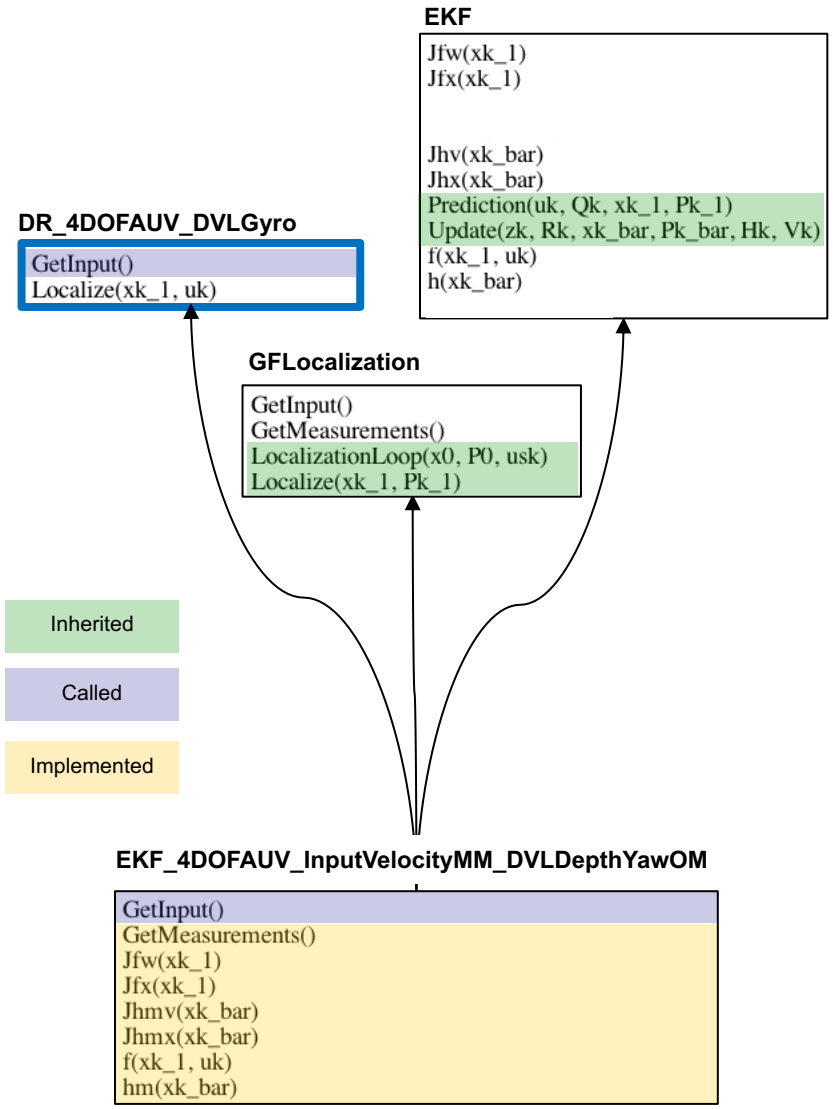
$$\underbrace{\begin{bmatrix} {}^N x_{USBL} \\ {}^N y_{USBL} \\ {}^N z_{USBL} \\ {}^N z_{depth} \\ {}^N \psi_{compass} \end{bmatrix}}_{\boldsymbol{z_k}} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\boldsymbol{H_k}} \underbrace{\begin{bmatrix} {}^N x_B \\ {}^N y_B \\ {}^N z_B \\ {}^N \psi_B \end{bmatrix}}_{\boldsymbol{x_k}} + \underbrace{\begin{bmatrix} v_{x_{USBL}} \\ v_{y_{USBL}} \\ v_{z_{USBL}} \\ v_{z_{depth}} \\ v_{\psi_{compass}} \end{bmatrix}}_{\boldsymbol{v_k}}$$

$$\boldsymbol{H_k} = \frac{\partial \boldsymbol{h}(\boldsymbol{x_k}, \boldsymbol{v_k})}{\partial \boldsymbol{x_k}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\boldsymbol{V_k} = \frac{\partial \boldsymbol{h}(\boldsymbol{x_k}, \boldsymbol{v_k})}{\partial \boldsymbol{v_k}} = \boldsymbol{I_{5\times 5}}$$

Gaussian Filters

Class Implementation



> We take the **GetInput()** method of the **Dead Reckoning** class returning the displacement.

Class *DR_4DOFAUV_DVLGyro : Localization*

Method `GetInput()`

$\boldsymbol{\nu_{DVL}} = ReadDVL();$

$\boldsymbol{r_{gyro}} = ReadGyro();$

$\boldsymbol{u_k} = \left[\boldsymbol{\nu_{DVL}^T} \quad \boldsymbol{r_{gyro}}\right]^T \cdot \Delta t$; **return** $\boldsymbol{u_k}$

Method `Localize`$(\boldsymbol{x_{k-1}}, \boldsymbol{u_k})$

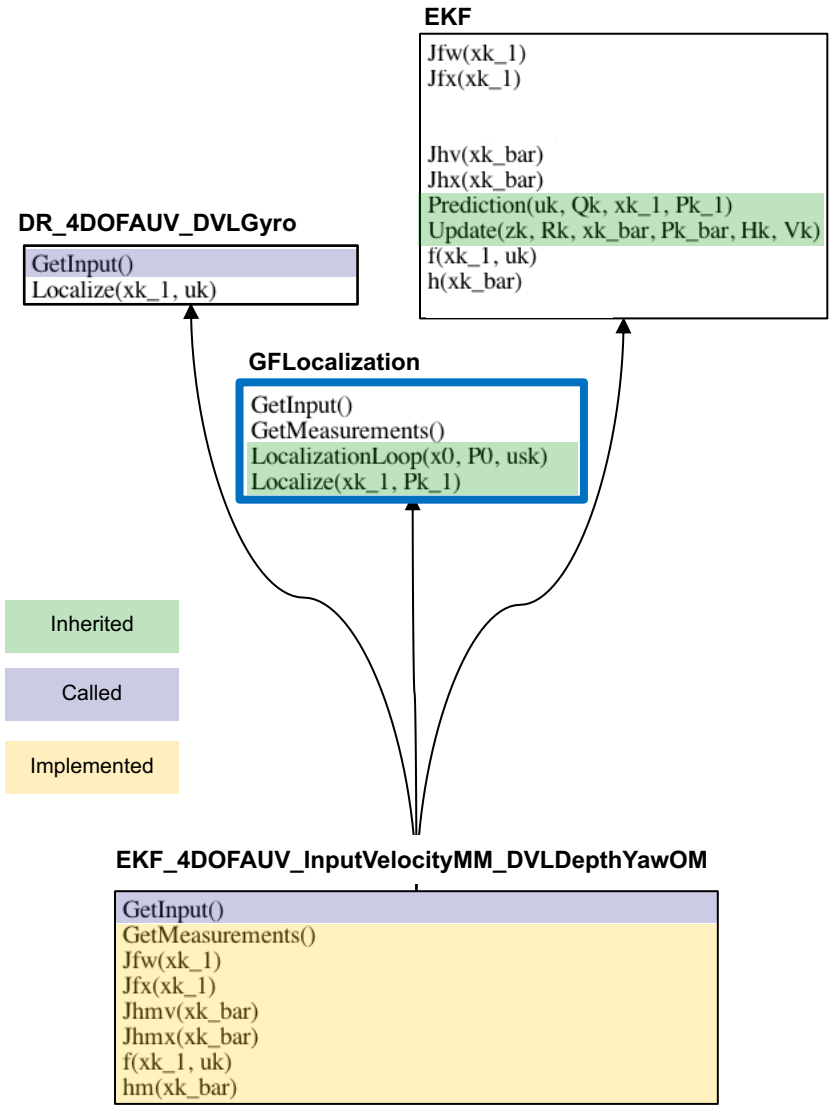
$\boldsymbol{x_k} = \boldsymbol{x_{k-1}} \oplus \boldsymbol{u_k};$

return $\boldsymbol{x_k}$;

Algorithm 1: DR_4DOFAUV_DVLGyro

Gaussian Filters

Class Implementation



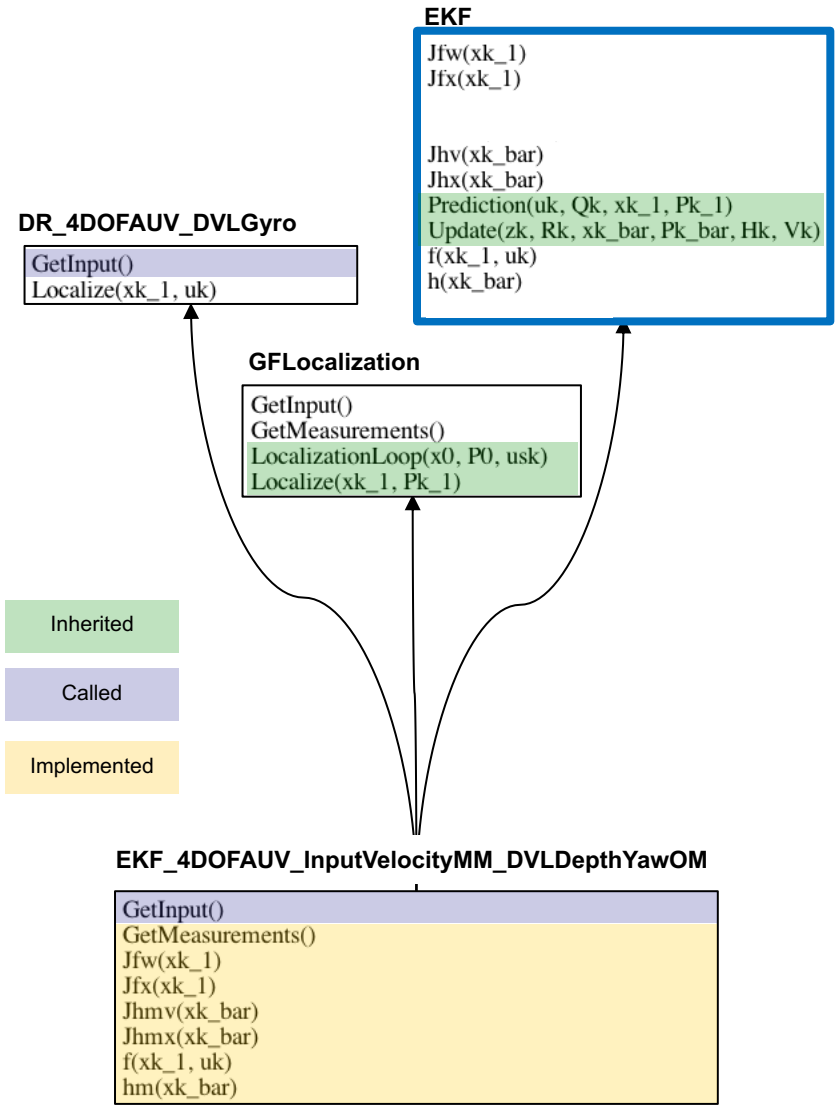
> We take the **LocalizationLoop** and **Localize** Method of the **GFLocalization** class.

Class *GFLocalization : Localization*

- **Pure Virtual** `GetInput()` $\rightarrow [\boldsymbol{u_k}, \boldsymbol{Q_k}]$;
- **Pure Virtual** `GetMeasurements()` $\rightarrow [\boldsymbol{z_k}, \boldsymbol{R_k}]$;
- **Method** `LocalizationLoop`$(\hat{\boldsymbol{x}}_0, \boldsymbol{P_0})$
  - $[\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}] = [\hat{\boldsymbol{x}}_0, \boldsymbol{P_0}]$;
  - **for** $k = 1$ *to* ∞ **do**
    - $[\hat{\boldsymbol{x}}_k, \boldsymbol{P_k}] = Localize(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
- **Method** `Localize`$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1})$
  - $[\boldsymbol{u_k}, \boldsymbol{Q_k}] = GetInput()$;
  - $[\bar{\hat{\boldsymbol{x}}}_k, \bar{\boldsymbol{P}}_k] = Prediction(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{u_k}, \boldsymbol{Q_k})$;
  - $[\boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k}] = GetMeasurements()$;
  - $[\hat{\boldsymbol{x}}_k, \bar{P}_k] = Update(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P}_{k-1}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
- **return** $[\hat{\boldsymbol{x}}_k, \boldsymbol{P_k}]$

Gaussian Filters

Class Implementation



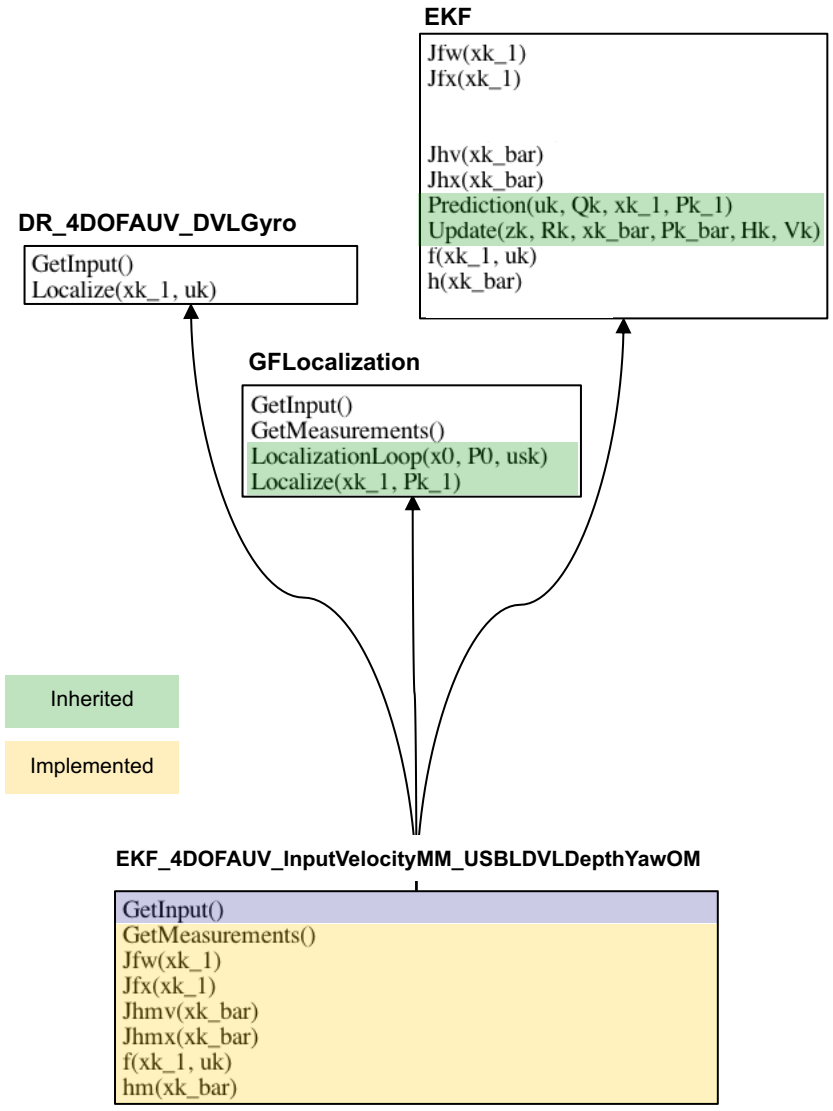
> We take the **Prediction** and **Update** Methods of the **EKF** class.

Class $EKF : KF$

- **Pure Virtual** $\texttt{h}(\hat{\boldsymbol{x}}_{\boldsymbol{k}}) \rightarrow \boldsymbol{h_k}$;
- **Pure Virtual** $\texttt{f}(\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{u}_k) \rightarrow \boldsymbol{f_k}$;
- **Pure Virtual** $J_{f_x}(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}) \rightarrow \boldsymbol{A_k}$; // Implements
  $$\boldsymbol{A_k} = \frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{x}_{k-1}}|_{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{w}_k=0}$$
- **Pure Virtual** $J_{f_w}(\hat{\boldsymbol{x}}_{\boldsymbol{k-1}}) \rightarrow \boldsymbol{W_k}$; // Implements
  $$\boldsymbol{W}_k = \frac{\partial \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k)}{\partial \boldsymbol{w}_k}|_{\boldsymbol{x}_{k-1}=\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{w}_k=0}$$
- **Pure Virtual** $J_{h_x}(\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}}) \rightarrow \boldsymbol{H_k}$; // Implements
  $$\boldsymbol{H}_k = \frac{\partial \boldsymbol{h}(\boldsymbol{x}_k, \boldsymbol{v}_k)}{\partial \boldsymbol{x}_k}|_{\boldsymbol{x}_k=\hat{\boldsymbol{x}}_k, \boldsymbol{w}_k=0}$$
- **Pure Virtual** $J_{h_v}(\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}}) \rightarrow \boldsymbol{V_k}$; // Implements
  $$\boldsymbol{V_k} = \frac{\partial \boldsymbol{h}(\boldsymbol{x_k}, \boldsymbol{v_k})}{\partial \boldsymbol{v_k}}|_{\boldsymbol{x}_k=\hat{\boldsymbol{x}}_k, \boldsymbol{w}_k=0}$$
- **Method** $\texttt{Prediction}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{P_{k-1}}, \boldsymbol{u}_k, \boldsymbol{Q_k})$
  - $\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}} = f(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k)$;
  - $\boldsymbol{A}_k = J_{f_x}(\hat{\boldsymbol{x}}_{k-1})$;
  - $\boldsymbol{W}_k = J_{f_w}(\hat{\boldsymbol{x}}_{k-1})$;
  - $\bar{\boldsymbol{P}}_k = \boldsymbol{A}_k \boldsymbol{P}_{k-1} \boldsymbol{A}_k^T + \boldsymbol{W}_k \boldsymbol{Q}_k \boldsymbol{W}_k^T$;
  - **return** $[\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}}, \bar{\boldsymbol{P}}_{\boldsymbol{k}}]$;
- **Method** $\texttt{Update}(\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}}, \hat{\bar{\boldsymbol{P}}}_{\boldsymbol{k}}, \boldsymbol{z_k}, \boldsymbol{R_k}, \boldsymbol{H_k}, \boldsymbol{V_k})$
  - $\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T (\boldsymbol{H}_k \bar{\boldsymbol{P}}_k \boldsymbol{H}_k^T + \boldsymbol{V}_k \boldsymbol{R}_k \boldsymbol{V}_k^T)^{-1}$;
  - $\hat{\boldsymbol{x}}_{\boldsymbol{k}} = \hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}} + \boldsymbol{K}_k \left(\boldsymbol{z}_k - \boldsymbol{h}(\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}})\right)$;
  - $\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k) \bar{\boldsymbol{P}}_k (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k)^T$;
  - **return** $[\hat{\boldsymbol{x}}_{\boldsymbol{k}}, \boldsymbol{P}_k]$;

Gaussian Filters

Class Implementation



> We implement all the **virtual methods** in the target **EKF Localization** class

Class *EKF_4DOFAUV_InputVelocityMM_USBLDVLDepthYawOM: GFLocalization, DR_4DOFAUV_DVLGyro, EKF*

Method `GetInput()`

$\boldsymbol{u_k} = DR_4DOFAUV_DVLGyro.GetInput()$; `// Displacement`

$$\boldsymbol{Q_k} = \begin{bmatrix} R_{DVL}\Delta t^2 & 0 \\ \mathbf{0_{3\times 3}} & \sigma^2_{gyro}\Delta t^2 \end{bmatrix} ; \quad \texttt{// Displacement uncertainty}$$

return $[\boldsymbol{u_k}\ \boldsymbol{Q_k}]$

Method `GetMeasurements()`

$[z_z, \sigma_z^2] = ReadDepth();$

$[z_\psi, \sigma_\psi^2] = ReadCompass();$

$[\boldsymbol{z_{USBL}}, \boldsymbol{R_{USBL}}] = ReadUSBL();$

$\boldsymbol{z_k} = [z_{USBL}\ z_z\ z_\psi]^T;$

$\boldsymbol{R_k} = blockdiag(\boldsymbol{R_{USBL}}\ diag(\sigma_z^2\ \sigma_\psi^2)^T);$

return $[\boldsymbol{z_k}\ \boldsymbol{R_k}]$

Method `f`$(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u_k})$

$\hat{\bar{\boldsymbol{x}}}_k = \hat{\boldsymbol{x}}_{k-1} \oplus \boldsymbol{u_k};$

return $\hat{\bar{\boldsymbol{x}}}_k;$

Motion Model

Method `Jfx`$(\hat{\boldsymbol{x}}_{k-1})$

return $\boldsymbol{J_{1\oplus}}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u_k});$

$$\left.\frac{\partial f(x_{k-1}, u_k, w_k)}{\partial w_k}\right|_{x_k=\hat{x}_k, w_k=0}$$

Method `Jfw`$(\hat{\boldsymbol{x}}_{k-1})$

return $\boldsymbol{J_{2\oplus}}(\hat{\boldsymbol{x}}_{k-1});$

$$\left.\frac{\partial f(x_{k-1}, u_k, w_k)}{\partial x_{k-1}}\right|_{x_k=\hat{x}_k, w_k=0}$$

Method `h`$(\hat{\boldsymbol{x}}_k)$

$\boldsymbol{H_x} = Jhx(\hat{\boldsymbol{x}}_k);$

return $\boldsymbol{H_x} \cdot \hat{\boldsymbol{x}}_k;$

Observation Model

Method `Jhx`$(\hat{\boldsymbol{x}}_{k-1})$

$$\boldsymbol{H_k} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

return $\boldsymbol{H_k};$

$$\left.\frac{\partial h(x_k, v_k)}{\partial x_k}\right|_{x_k=\hat{x}_k, v_k=0}$$

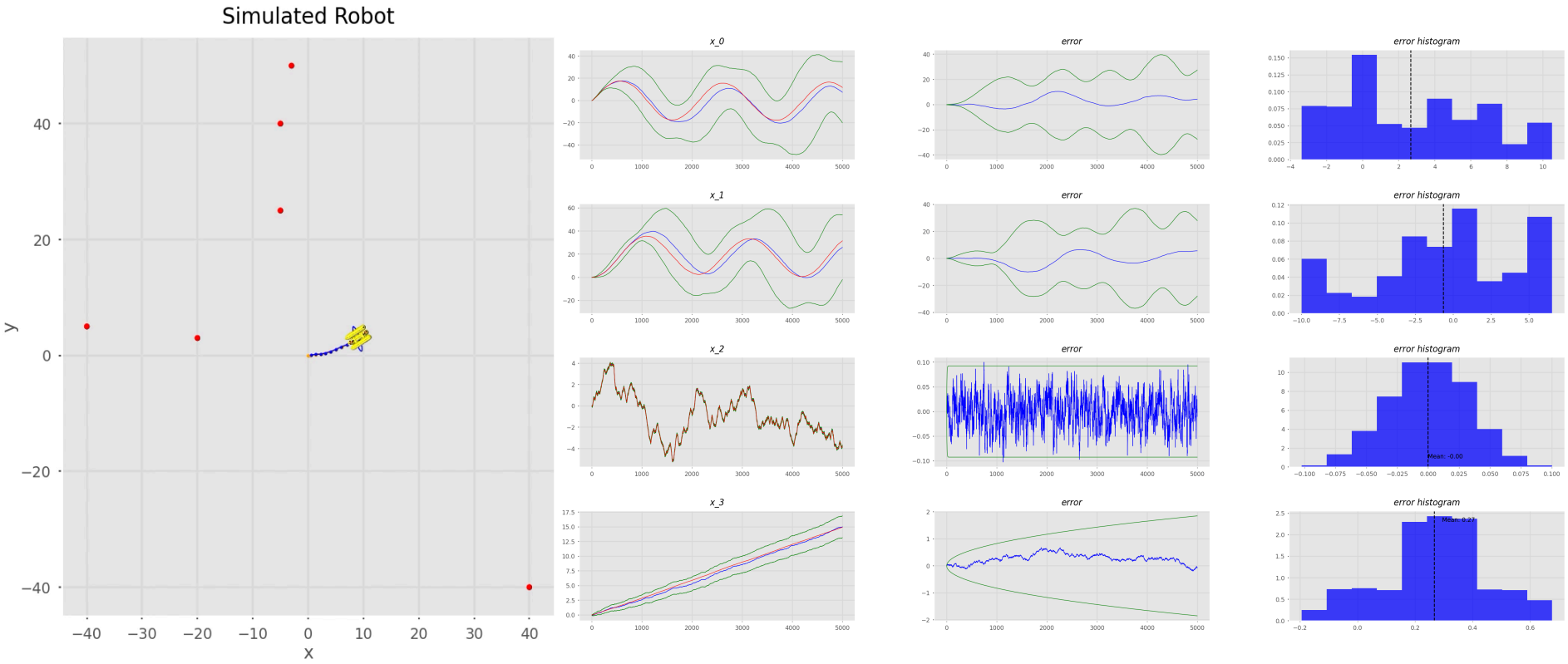
Method `Jhv`$(\hat{\boldsymbol{x}}_{k-1})$

$\boldsymbol{V_k} = \boldsymbol{I_{5\times 5}}$

return $\boldsymbol{V_k};$

$$\left.\frac{\partial h(x_k, v_k)}{\partial v_k}\right|_{x_k=\hat{x}_k, v_k=0}$$

Gaussian Filters



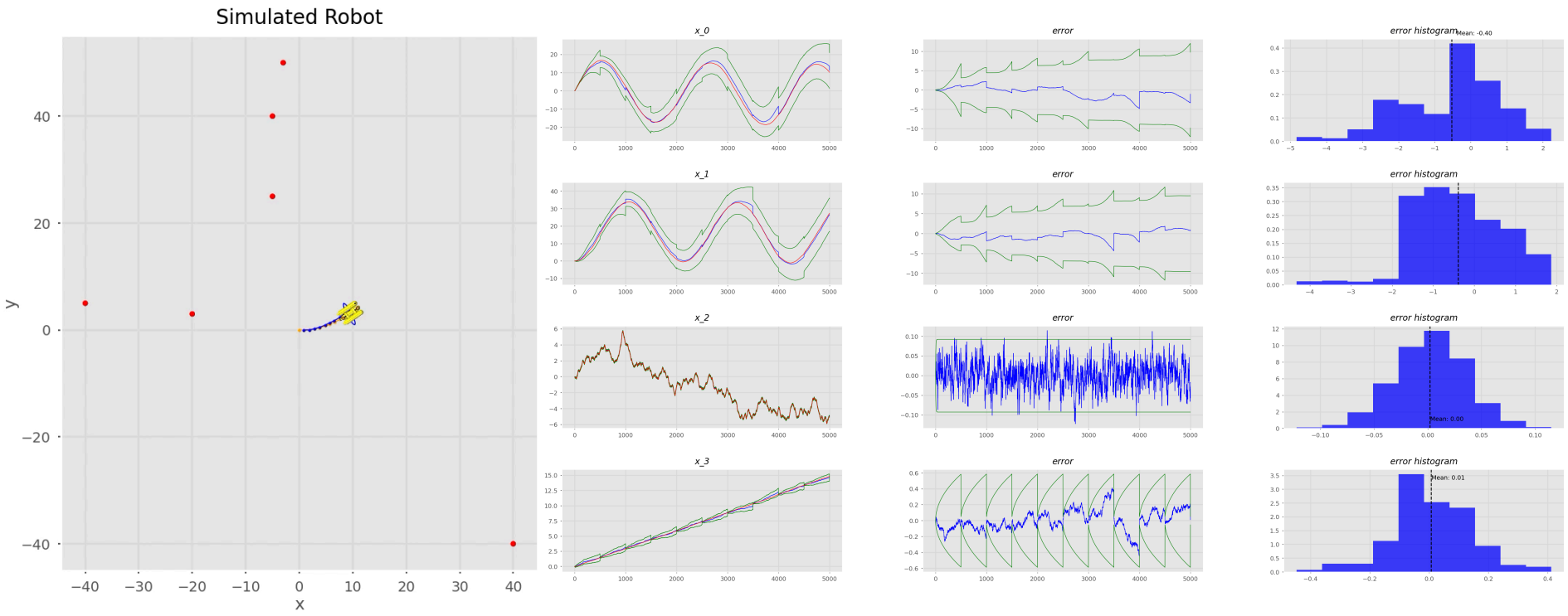
Input

- **DVL:** 1 every 0.1 s, $R_{DVL}=diag([0.1^2,0.1^2,0.1^2])$
- **Gyro:** 1 every 0.1 s, $\sigma_{yaw}= 5º/s$

Measurement

- **Depth:** 1 every 0.1 s, $\sigma_{depth}= 0.1$ m
- **Compass:** None, $\sigma_{yaw}= 1º$
- **USBL:** None, $R_{USBL}=diag([2^2\,m^2,5^2\,deg^2,5^2\,deg^2])$

Gaussian Filters



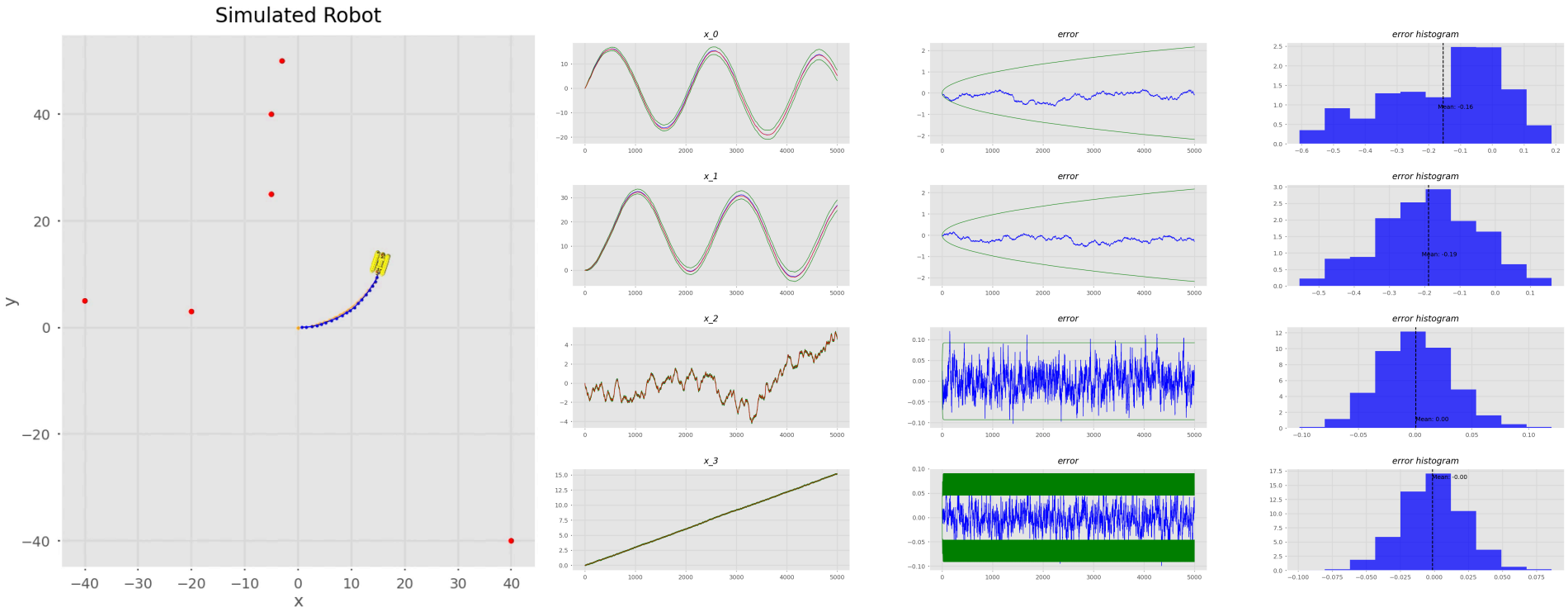
Input

- **DVL:** 1 every 0.1 s, $R_{DVL}=diag([0.1^2,0.1^2,0.1^2])$
- **Gyro:** 1 every 0.1 s, $\sigma_{yaw}= 5º/s$

Measurement

- **Depth:** 1 every 0.1 s, $\sigma_{depth}= 0.1$ m
- **Compass:** 1 every 50 s, $\sigma_{yaw}= 1º$
- **USBL:** None, $R_{USBL}=diag([2^2\,m^2,5^2\,deg^2,5^2\,deg^2])$

Gaussian Filters



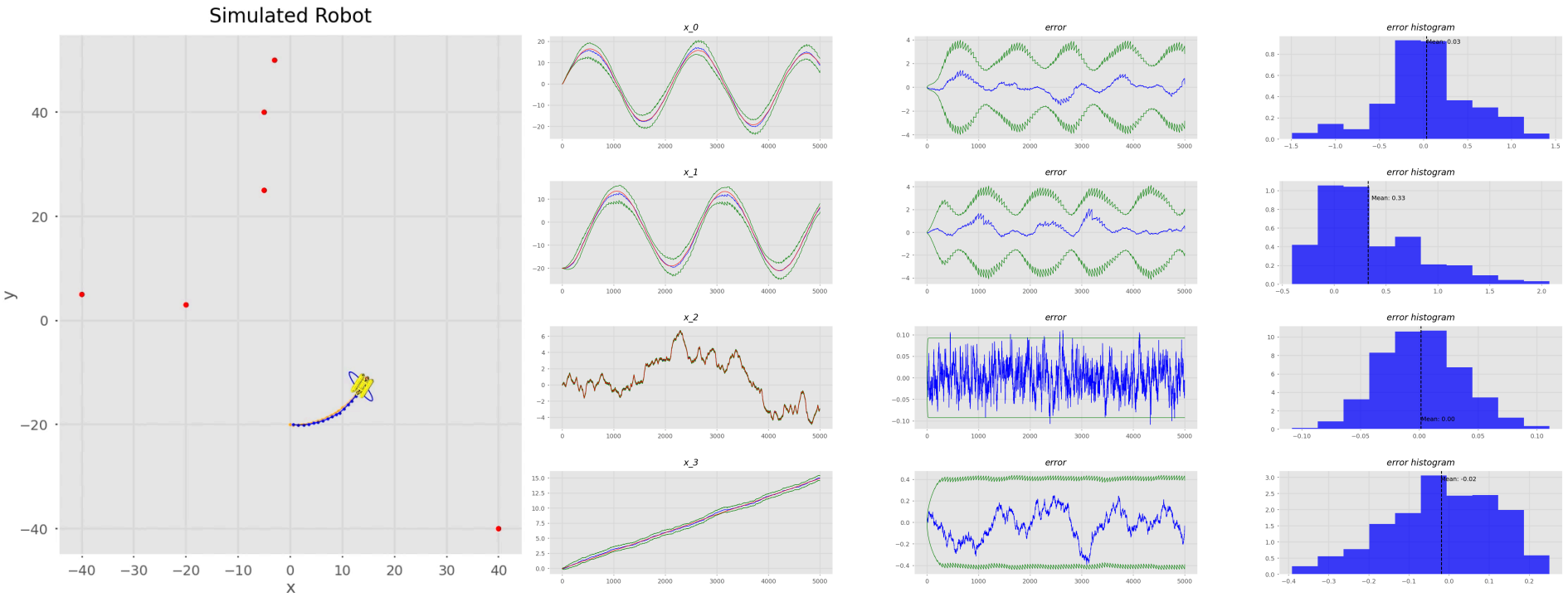
Input

- **DVL:** 1 every 0.1 s, $R_{DVL}=diag([0.1^2,0.1^2,0.1^2])$
- **Gyro:** 1 every 0.1 s, $\sigma_{yaw}= 5º/s$

Measurement

- **Depth:** 1 every 0.1 s, $\sigma_{depth}= 0.1$ m
- **Compass:** 1 every s, $\sigma_{yaw}= 1º$
- **USBL:** None, $R_{USBL}=diag([2^2\,m^2,5^2\,deg^2,5^2\,deg^2])$

Gaussian Filters



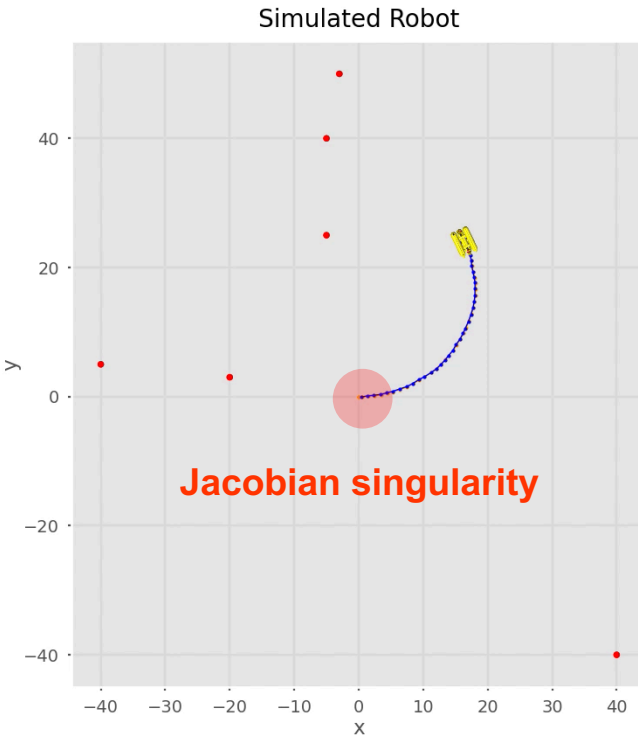
Input

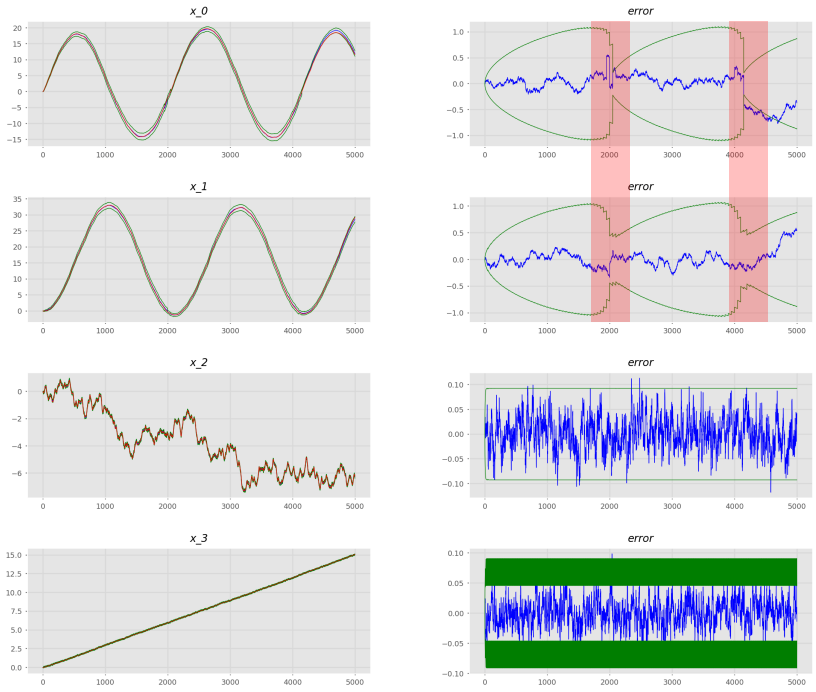
- **DVL:** 1 every 0.1 s, $R_{DVL}=diag([0.1^2,0.1^2,0.1^2])$
- **Gyro:** 1 every 0.1 s, $\sigma_{yaw}= 5º/s$

Measurement

- **Depth:** 1 every 0.1 s, $\sigma_{depth}= 0.1$ m
- **Compass:** None, $\sigma_{yaw}= 1º$
- **USBL:** 1 every 5 s, $R_{USBL}=diag([2^2\,m^2,5^2\,deg^2,5^2\,deg^2])$

Gaussian Filters





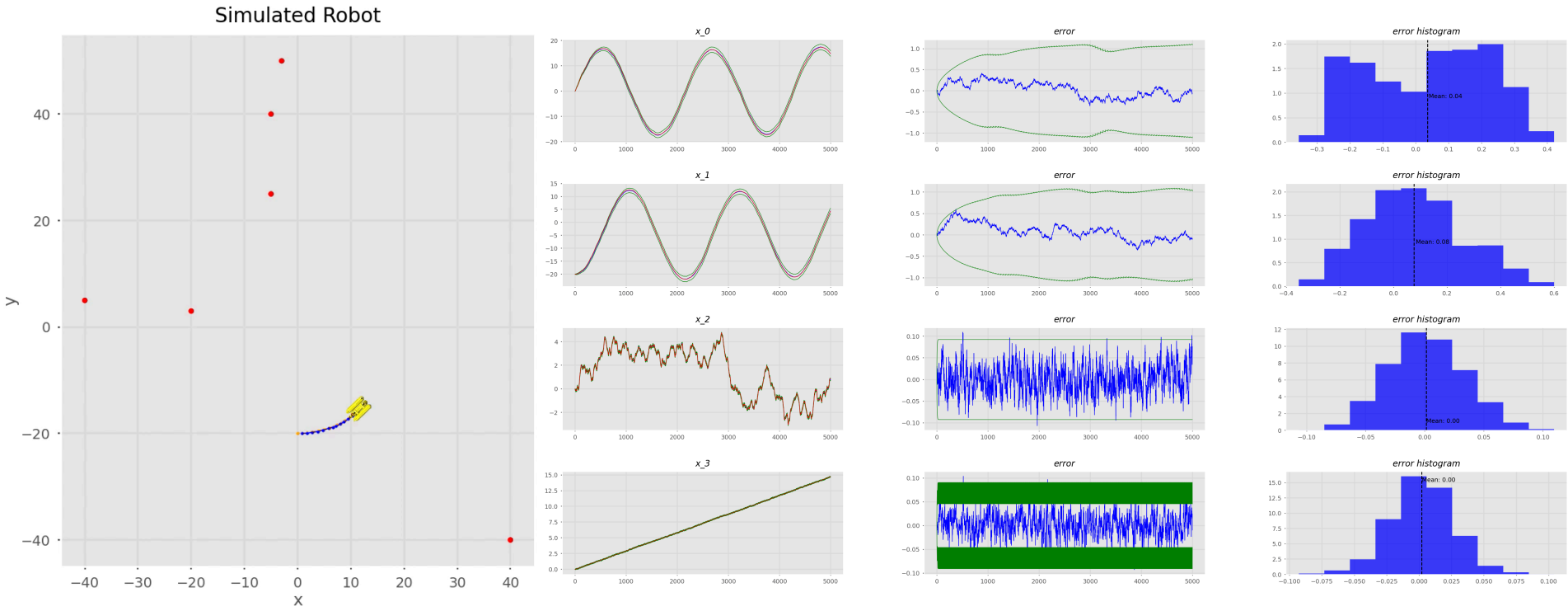
Input

- **DVL:** 1 every 0.1 s, $R_{DVL}=diag([0.1^2,0.1^2,0.1^2])$
- **Gyro:** 1 every 0.1 s, $\sigma_{yaw}= 5º/s$

Measurement

- **Depth:** 1 every 0.1 s, $\sigma_{depth}= 0.1$ m
- **Compass:** 1 every s, $\sigma_{yaw}= 1º$
- **USBL:** 1 every 5 s, $R_{USBL}=diag([2^2\,m^2,5^2\,deg^2,5^2\,deg^2])$

Gaussian Filters



Input

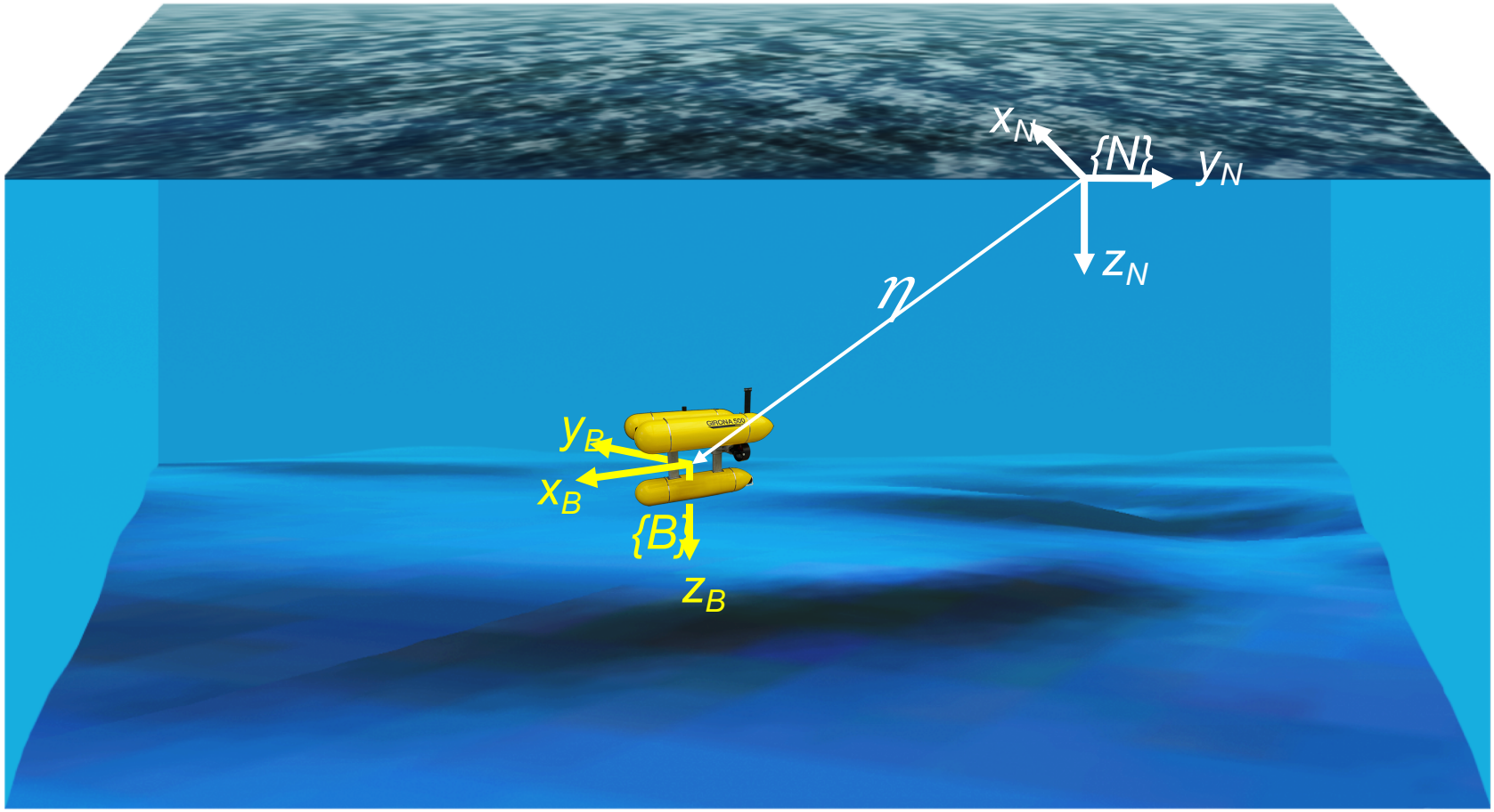
- **DVL:** 1 every 0.1 s, $R_{DVL}=diag([0.1^2,0.1^2,0.1^2])$
- **Gyro:** 1 every 0.1 s, $\sigma_{yaw}= 5º/s$

Measurement

- **Depth:** 1 every 0.1 s, $\sigma_{depth}= 0.1$ m
- **Compass:** 1 every s, $\sigma_{yaw}= 1º$
- **USBL:** 1 every 5 s, $R_{USBL}=diag([2^2\,m^2,5^2\,deg^2,5^2\,deg^2])$

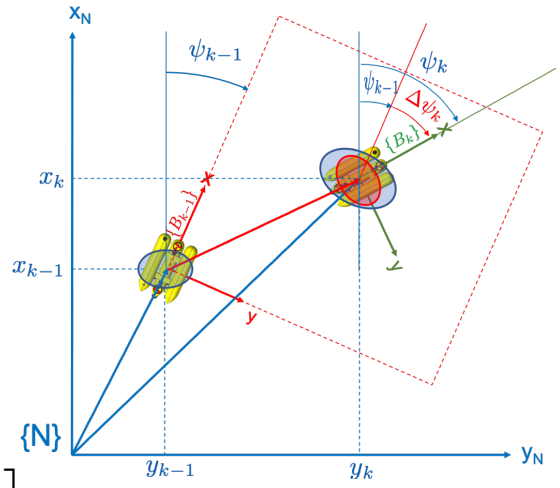
Gaussian Filters

Example: AUV Localization using a Ct Velocity Model, DVL & Compass



Gaussian Filters

AUV Localization using a Ct Velocity Model, Depth, Compass & DVL



Vector State

$$\boldsymbol{x}_k = [\boldsymbol{\eta}_k^T \boldsymbol{\nu}_k^T]^T \sim \mathcal{N}([\hat{\boldsymbol{\eta}}_k^T \hat{\boldsymbol{\nu}}_k^T]^T, \boldsymbol{P}_k)$$

$$\boldsymbol{\eta}_k^T =^N [x_k \;\; y_k \;\; z_k \;\; \psi_k]^T$$

$$\boldsymbol{\nu}_k^T =^B [u_k \;\; v_k \;\; w_k \;\; r_k]^T$$

Motion Model

$$\bar{\boldsymbol{x}}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{w}_k) = \begin{bmatrix} \boldsymbol{\eta_k} \\ \boldsymbol{\nu_k} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\eta_{k-1}} \oplus \left(\boldsymbol{\nu_{k-1}} \Delta t + \boldsymbol{w_k} \frac{\Delta t^2}{2} \right) \\ \boldsymbol{\nu_{k-1}} + \boldsymbol{w_k} \Delta t \end{bmatrix}$$

$$\left. \frac{\partial \boldsymbol{f}(\boldsymbol{x_{k-1}}, \boldsymbol{u_k}, \boldsymbol{w_k})}{\partial \boldsymbol{x_{k-1}}} \right|_{\boldsymbol{x_{k-1}} = \hat{\boldsymbol{x}}_{k-1}, \hat{\boldsymbol{w}}_k = \boldsymbol{0}} = \begin{bmatrix} \boldsymbol{J_{1\oplus}}(\hat{\boldsymbol{x}}_{k-1}) & \boldsymbol{J_{2\oplus}}(\hat{\boldsymbol{x}}_{k-1}) \Delta t \\ \boldsymbol{0_{4\times 4}} & \boldsymbol{I_{4\times 4}} \end{bmatrix};$$

$$\left. \frac{\partial \boldsymbol{f}(\boldsymbol{x_{k-1}}, \boldsymbol{u_k}, \boldsymbol{w_k})}{\partial \boldsymbol{w_k}} \right|_{x_{k-1} = \hat{\boldsymbol{x}}_{k-1}, \hat{\boldsymbol{w}}_k = \boldsymbol{0}} = \begin{bmatrix} \frac{\boldsymbol{J_{2\oplus}}(\hat{\boldsymbol{x}}_{k-1}) \Delta t^2}{2} \\ \boldsymbol{I_{4\times 4}} \Delta t \end{bmatrix};$$

Observation Model

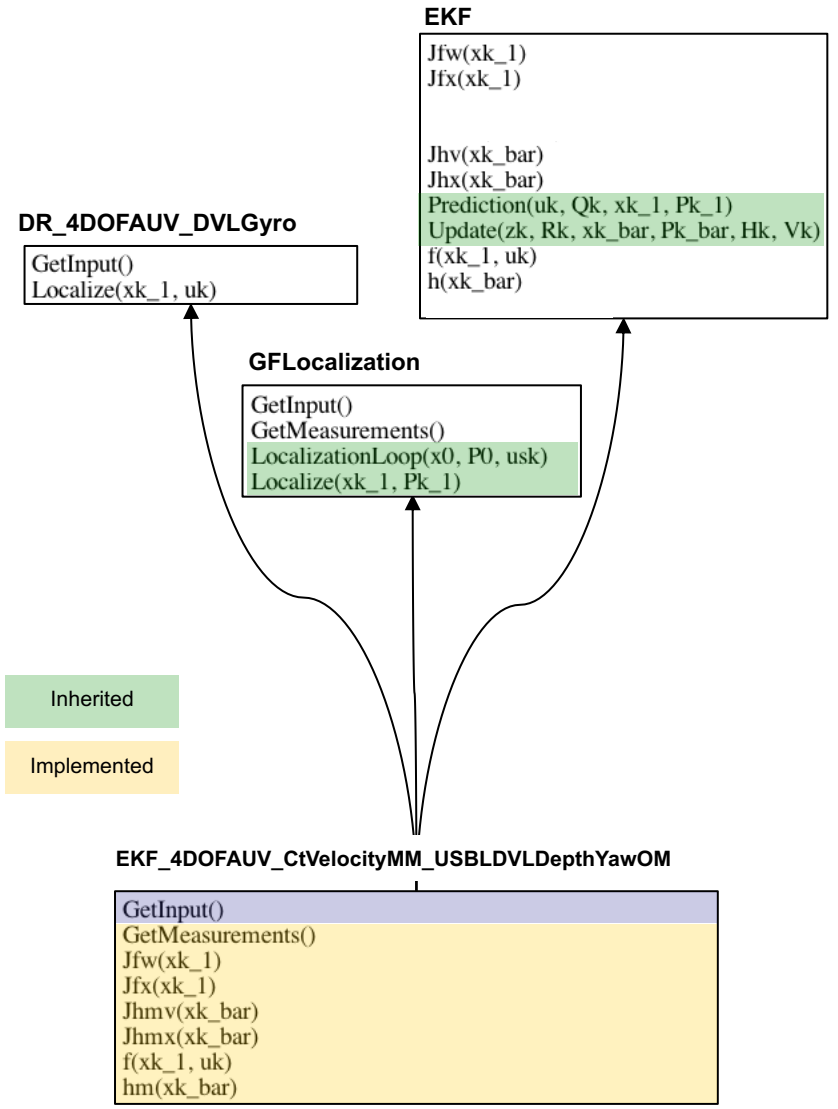
$$\boldsymbol{z}_k = \boldsymbol{h}(\boldsymbol{x}_k, \boldsymbol{v}_k)$$

$$\boldsymbol{v}_k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_k)$$

$$\overbrace{\begin{bmatrix} \psi_{Compass} \\ u_{DVL} \\ v_{DVL} \\ w_{DVL} \end{bmatrix}}^{\boldsymbol{z_k}} = \overbrace{\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}}^{\boldsymbol{H_k}} \overbrace{\begin{bmatrix} x_k \\ y_k \\ z_k \\ \psi_k \\ u_k \\ v_k \\ w_k \\ r_k \end{bmatrix}}^{\boldsymbol{x_k}} + \overbrace{\begin{bmatrix} v_{\psi_{Compass}} \\ v_{u_{DVL}} \\ v_{v_{DVL}} \\ v_{w_{DVL}} \end{bmatrix}}^{\boldsymbol{v_k}}$$

Gaussian Filters

Class Implementation



Class *EKF_4DOFAUV_CtVelocityMM_USBLDVLDepthYawOM:*
GFLocalization: GFLocalization, DR_4DOFAUV_DVLGyro, EKF

Method `GetInput()`

$\boldsymbol{u_k} = [\,]$; // empty matrix

$\boldsymbol{Q_k} = diag([\sigma_{\dot{u}}^2\ \sigma_{\dot{v}}^2\ \sigma_{\dot{w}}^2\ \sigma_{\dot{r}}^2\ ]^T)$; // accelleration uncertainty

return $[\boldsymbol{u_k}\ \boldsymbol{Q_k}]$

Method `GetMeasurements()`

$[z_z, \sigma_z^2] = ReadDepth();$

$[z_\psi, \sigma_\psi^2] = ReadCompass();$

$[\boldsymbol{z_{DVL}}, \boldsymbol{R_{DVL}}] = ReadDVL();$

$\boldsymbol{z_k} = [z_z\ z_\psi\ \boldsymbol{z_{DVL}}^T]^T;$

$\boldsymbol{R_k} = blockdiag(diag(\sigma_z^2, \sigma_\psi^2), \boldsymbol{R_{DVL}});$

return $[\boldsymbol{z_k}\ \boldsymbol{R_k}]$

Method `f(`$\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u_k}$`)`

$\begin{bmatrix}\hat{\boldsymbol{\eta}}_{k-1}\\ \hat{\boldsymbol{\nu}}_{k-1}\end{bmatrix} = \hat{\boldsymbol{x}}_{k-1};$

$\hat{\bar{\boldsymbol{x}}}_{\boldsymbol{k}} = \begin{bmatrix}\hat{\boldsymbol{\eta}}_{k-1} \oplus \hat{\boldsymbol{\nu}}_{k-1}\Delta t\\ \hat{\boldsymbol{\nu}}_{k-1}\end{bmatrix};$

return $\boldsymbol{x_k}$;

Motion Model

Method `Jfx(`$\hat{\boldsymbol{x}}_{k-1}$`)`

$\boldsymbol{J} = \begin{bmatrix}\boldsymbol{J_{1\oplus}}(\hat{\boldsymbol{\eta}}_{k-1}, \hat{\boldsymbol{\nu}}_{k-1}\boldsymbol{\Delta t}) & \boldsymbol{J_{2\oplus}}(\hat{\boldsymbol{\eta}}_{k-1}, \hat{\boldsymbol{\nu}}_{k-1}\boldsymbol{\Delta t})\Delta t\\ \boldsymbol{0_{4\times4}} & \boldsymbol{I_{4\times4}}\end{bmatrix};$

return $\boldsymbol{J}$

$\left.\frac{\partial f(x_{k-1}, u_k, w_k)}{\partial w_k}\right|_{x_k=\hat{x}_k, w_k=0}$

Method `Jfw(`$\boldsymbol{x}_{k-1}$`)`

$\boldsymbol{J} = \begin{bmatrix}\frac{\boldsymbol{J_{2\oplus}}(\hat{\boldsymbol{\eta}}_{k-1}, \hat{\boldsymbol{\nu}}_{k-1}\boldsymbol{\Delta t})\Delta t}{2}\\ \boldsymbol{I_{4\times4}}\end{bmatrix};$

return $\boldsymbol{J}$

$\left.\frac{\partial f(x_{k-1}, u_k, w_k)}{\partial x_{k-1}}\right|_{x_k=\hat{x}_k, w_k=0}$

Observation Model

Method `h(`$\hat{\boldsymbol{x}}_{\boldsymbol{k}}$`)`

$\boldsymbol{H_k} = Jhx(\hat{\boldsymbol{x}}_{\boldsymbol{k}});$

return $\boldsymbol{H_k}\hat{\boldsymbol{x}}_{\boldsymbol{k}}$

Method `Jhx(`$\hat{\boldsymbol{x}}_{k-1}$`)`

$\boldsymbol{H_k} = \begin{bmatrix}0&0&1&0&0&0&0&0\\0&0&0&1&0&0&0&0\\0&0&0&0&1&0&0&0\\0&0&0&0&0&1&0&0\\0&0&0&0&0&0&1&0\end{bmatrix}$

return $\boldsymbol{H_k}$;

$\left.\frac{\partial h(x_k, v_k)}{\partial x_k}\right|_{x_k=\hat{x}_k, v_k=0}$

Method `Jfw(`$\hat{\boldsymbol{x}}_{k-1}$`)`

$\boldsymbol{V_k} = \boldsymbol{I_{5\times5}}$

return $\boldsymbol{V_k}$;

$\left.\frac{\partial h(x_k, v_k)}{\partial v_k}\right|_{x_k=\hat{x}_k, v_k=0}$